

串

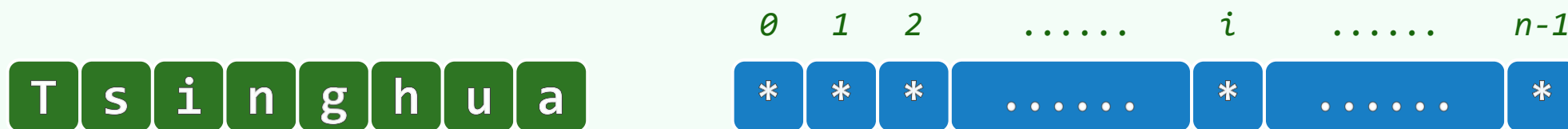
ADT

13-A

邓俊辉

deng@tsinghua.edu.cn

术语



$$S.substr(i, k) = S[i, i + k), \quad 0 \leq i < n, \quad 0 \leq k$$



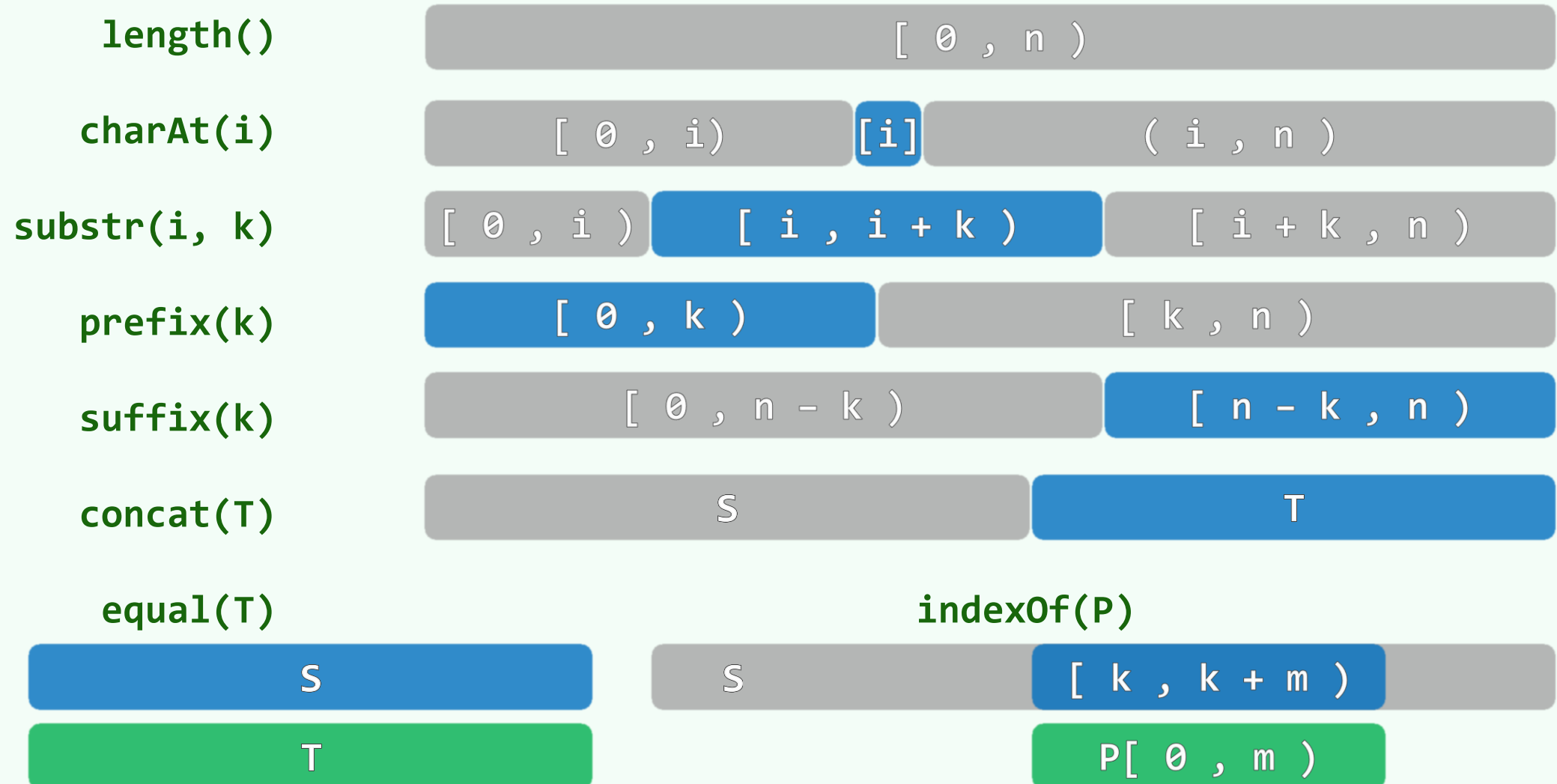
$$S.prefix(k) = S.substr(0, k) = S[0, k), \quad 0 \leq k \leq n$$



$$S.suffix(k) = S.substr(n - k, k) = S[n - k, n), \quad 0 \leq k \leq n$$



$$S.substr(i, k) = S.prefix(i + k).suffix(k) = S.suffix(n - i).prefix(k)$$



实例

- ❖ `"data structures".length() = 15` `"data structures".charAt(5) = 's'`
`"data structures".prefix(4) = "data"` `"data structures".suffix(10) = "structures"`
`"data structures".concat(" & algorithms") = "data structures & algorithms"`
`"algorithms".equal("data structures") = false`
`"data structures and algorithms".indexOf("string") = -1`
`"data structures and algorithms".indexOf("algorithm") = 20`
- ❖ **<string.h>中的对应功能:** `strlen()`、`strcpy()`、`strcat()`、`strcmp()`、`strstr()`
- ❖ 以下, 直接利用**字符数组**实现字符串, 转而重点讨论**串匹配算法**

串

模式匹配：问题 & 蛮力算法

13-B

一切事情，一旦失败，就显得很愚蠢

长的是磨难，短的是人生

邓俊辉

deng@tsinghua.edu.cn

循模式访问：问题特点 + 测试方法

Text : a man with money is no match against a man on a mission

Pattern : match

random T + random P ?

random T + substring P !

$$n = |T| \quad m = |P|$$

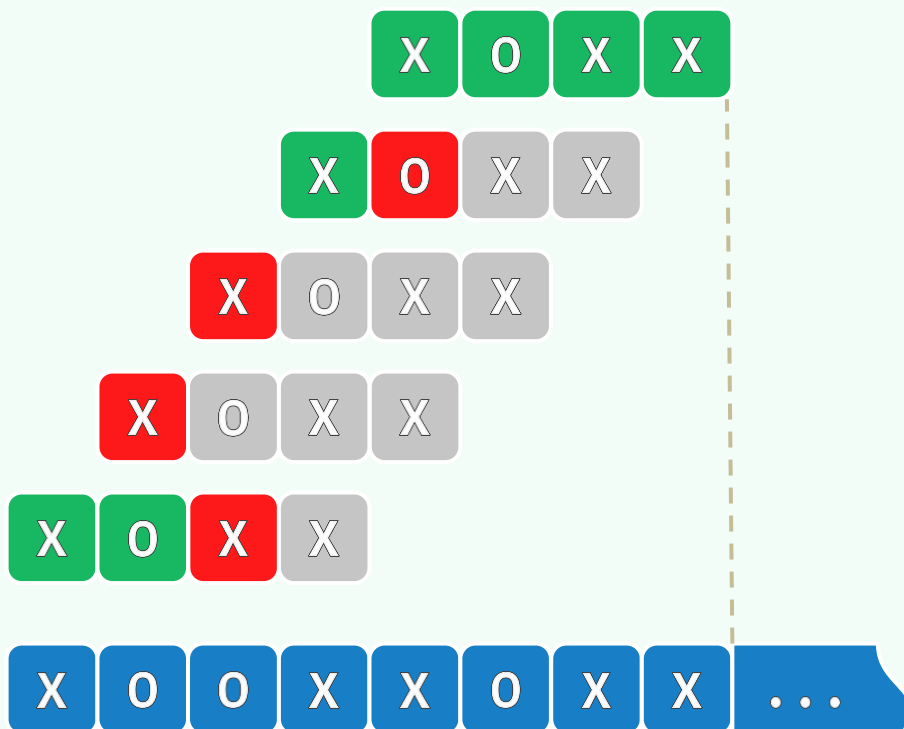
$$2 \ll m \ll n$$

$$s = |\Sigma|$$

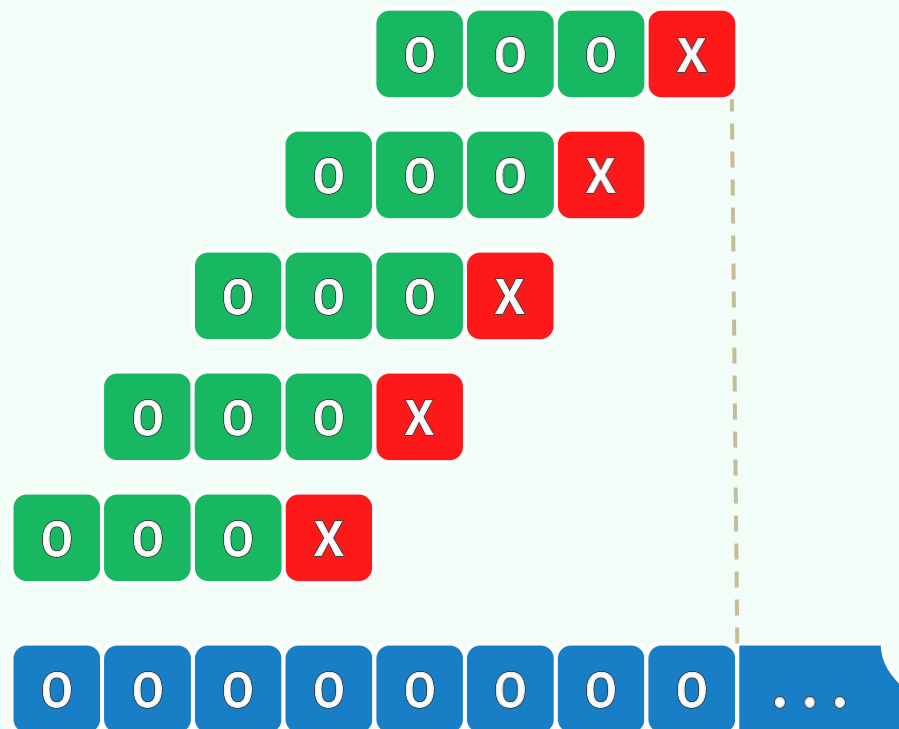
$$Pr_{match} \approx n/s^m \rightarrow 0$$

蛮力策略

Best : $\Omega(n)$



Worst : $\mathcal{O}(n \cdot m)$



版本1

```
int match( char * P, char * T ) {
```

```
    int n = strlen(T), i = 0;
```

```
    int m = strlen(P), j = 0;
```

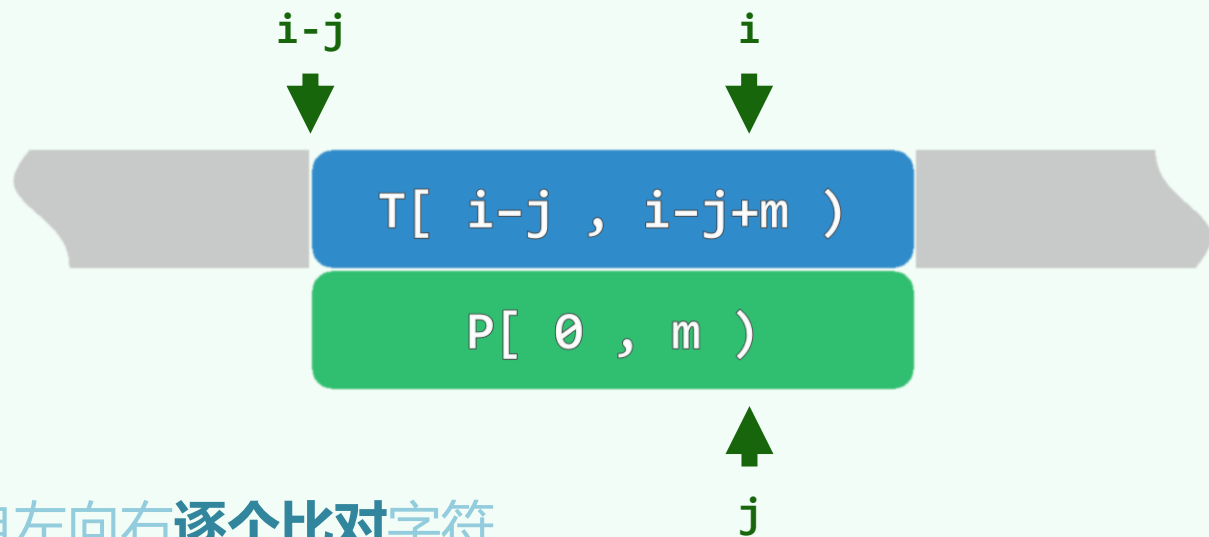
```
    while ( (j < m) && (i <= n-m) ) //自左向右逐个比对字符
```

```
        if ( T[i] == P[j] ) { i++; j++; } //若匹配，则转到下一对字符
```

```
        else { i -= j-1; j = 0; } //否则，T回退、P复位
```

```
    return i - j; //最终的对齐位置：藉此足以判断匹配结果
```

```
}
```



版本2

```
int match( char * P, char * T ) {
```

```
    int n = strlen(T), i = 0;
```

```
    int m = strlen(P), j = 0;
```

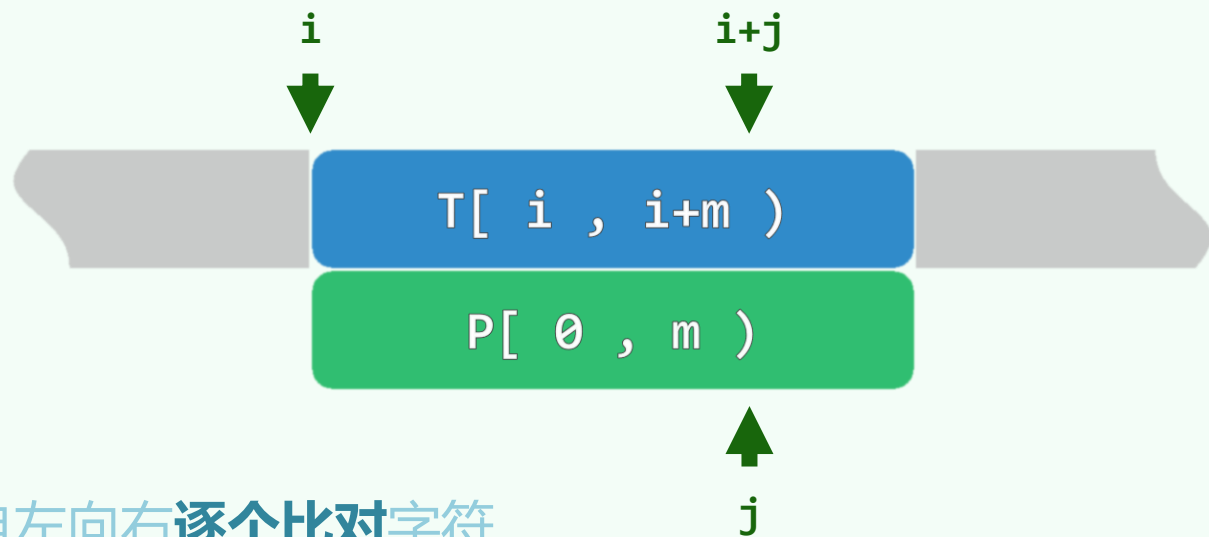
```
    while ( (j < m) && (i <= n-m) ) //自左向右逐个比对字符
```

```
        if ( T[i] == P[j] ) { j++; } //若匹配，则转到下一对字符
```

```
        else { i++; j = 0; } //否则，T回退、P复位
```

```
    return i; //最终的对齐位置：藉此足以判断匹配结果
```

```
}
```



13-C1

串

KMP算法：记忆法

知易者不占，善易者不蓍

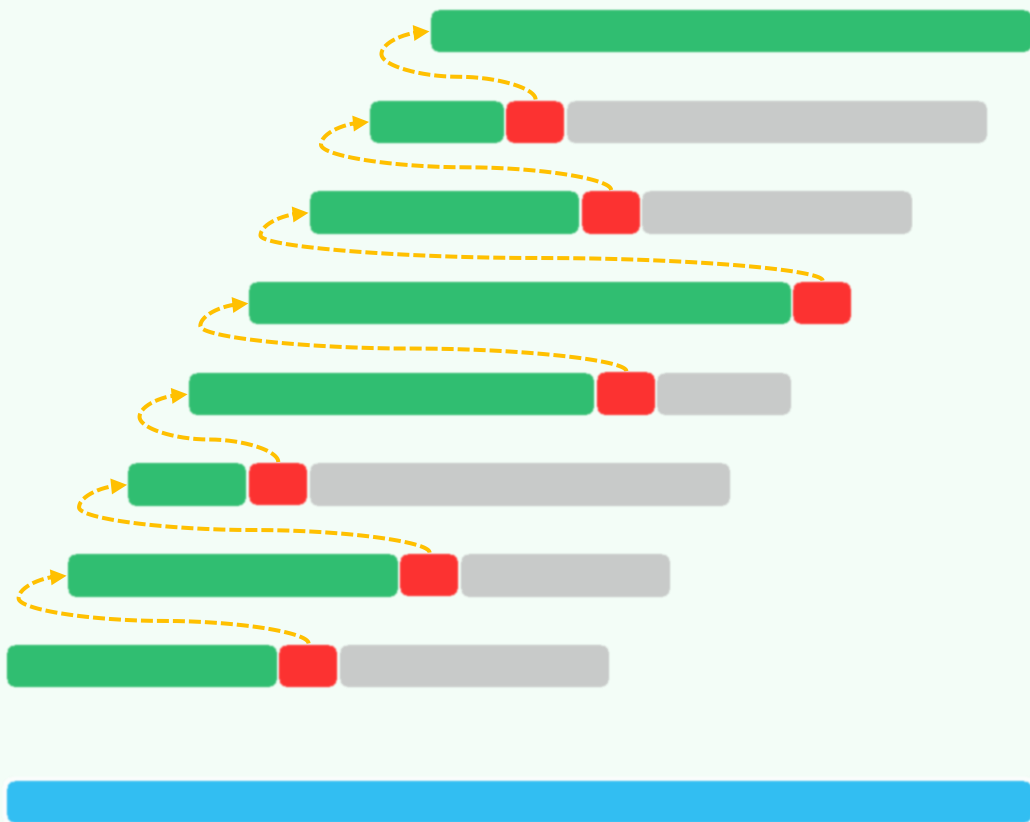
只有记忆才能建立起身份，即你个人的相同性

丧失了记忆又不自知，那才是人生最快乐的时光

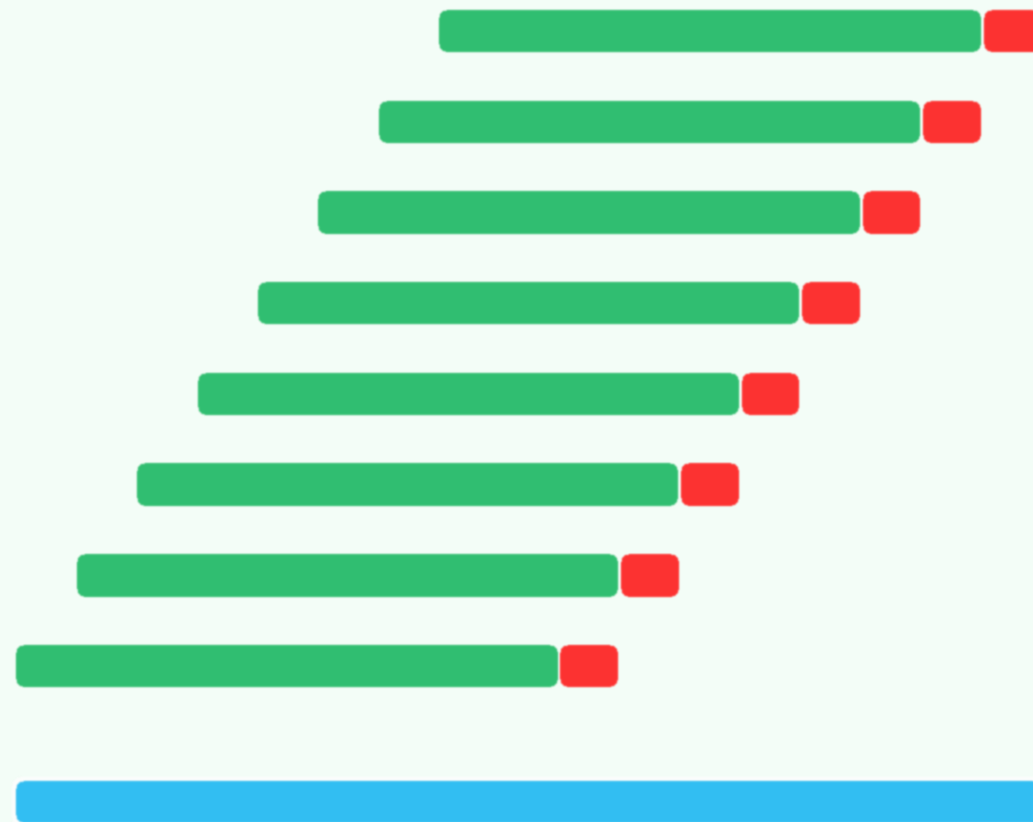
邓俊辉

deng@tsinghua.edu.cn

低效率：无视局部匹配



Worst : $O(n \cdot m)$



不变性：“超强大脑”玩“记忆翻牌消消看”

❖ 在任一时刻，都有

$$T[i-j, i) == P[0, j)$$

❖ 亦即，我们业已掌握 $T[i-j, i)$ 的所有信息

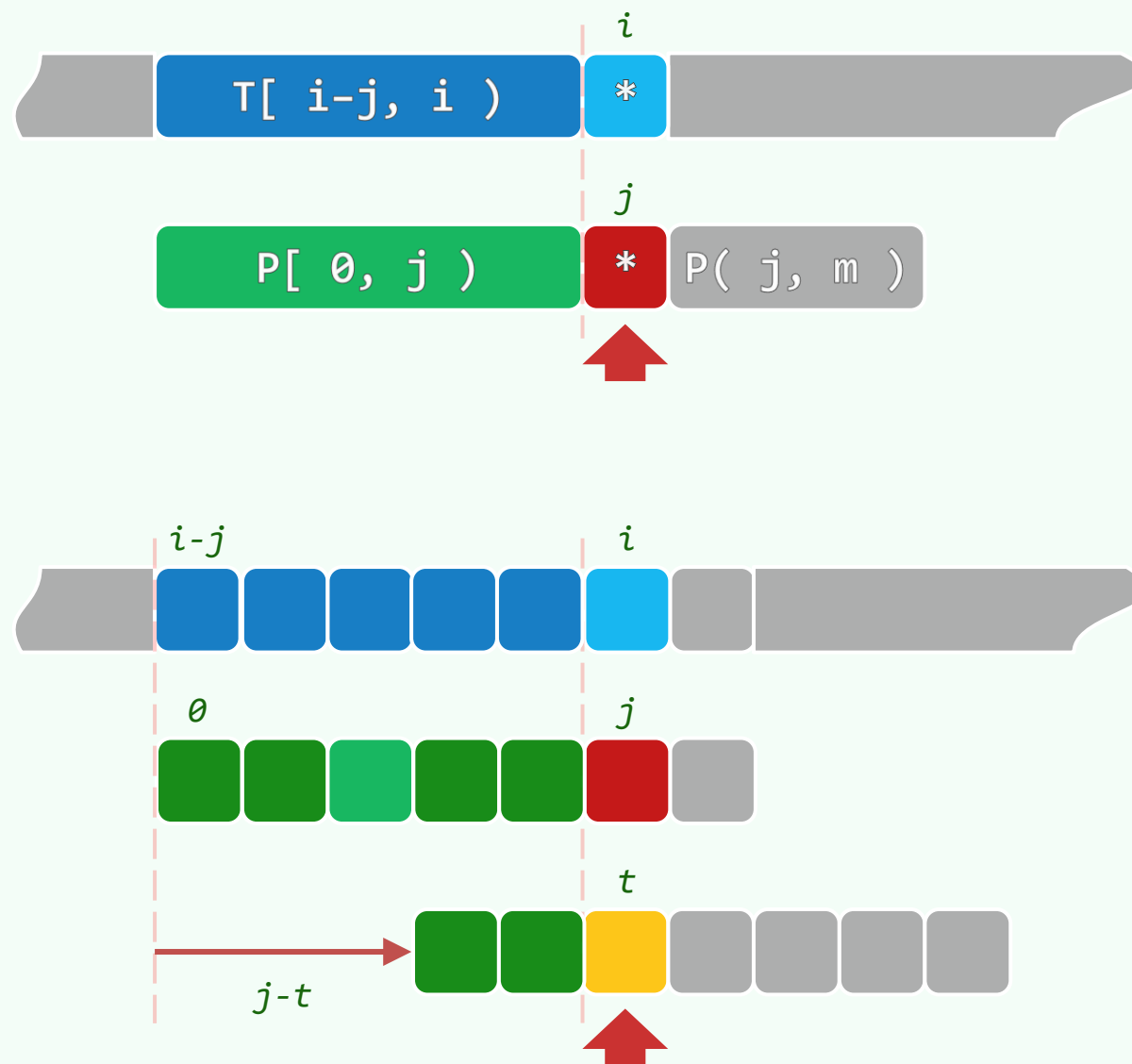
既如此...

❖ 一旦失败，我们就应已知

哪些位置**值得/不必**对齐

而且...在下一轮比对中...

❖ $T[i-t, i)$ 可**径直接受**，而不必再做比对



过目不忘？凡事预则立，不预则废！

❖ 如此， i 将永远不必回退！

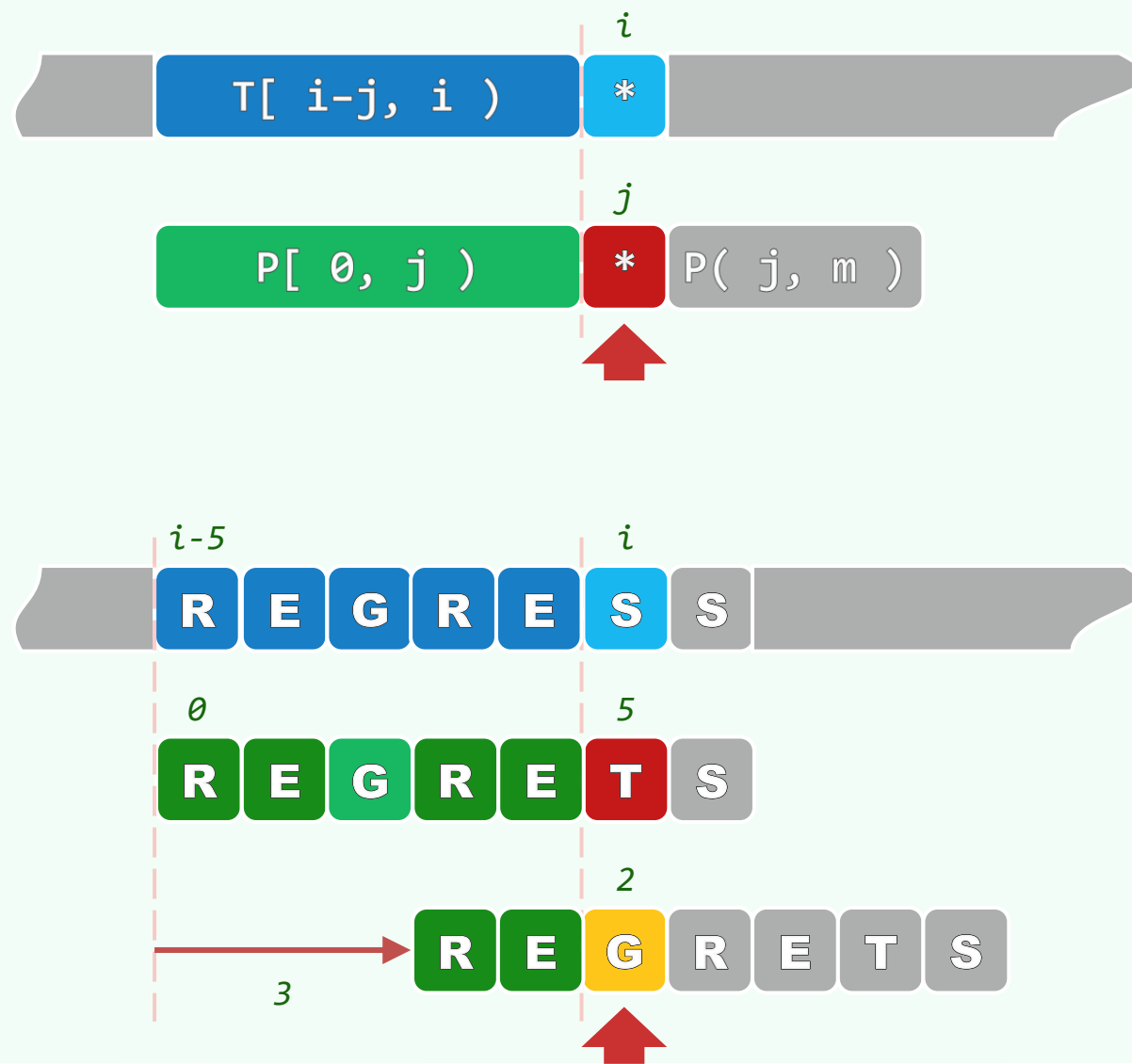
- 比对成功，则与 j 同步前进一个字符
- 否则， j 更新为某更小的 t ，并继续比对

❖ 即便是更为复杂的情况，依然可行

❖ 优化 = P 可快速右移 + 避免重复比对

❖ 为确定 t ，需花费多少时间和空间？

更重要地，可否在事先就确定？



串

KMP算法：查询表

13-C2

邓俊辉

deng@tsinghua.edu.cn

好记性不如烂笔头

The farther backward you can look, the farther forward you can see.

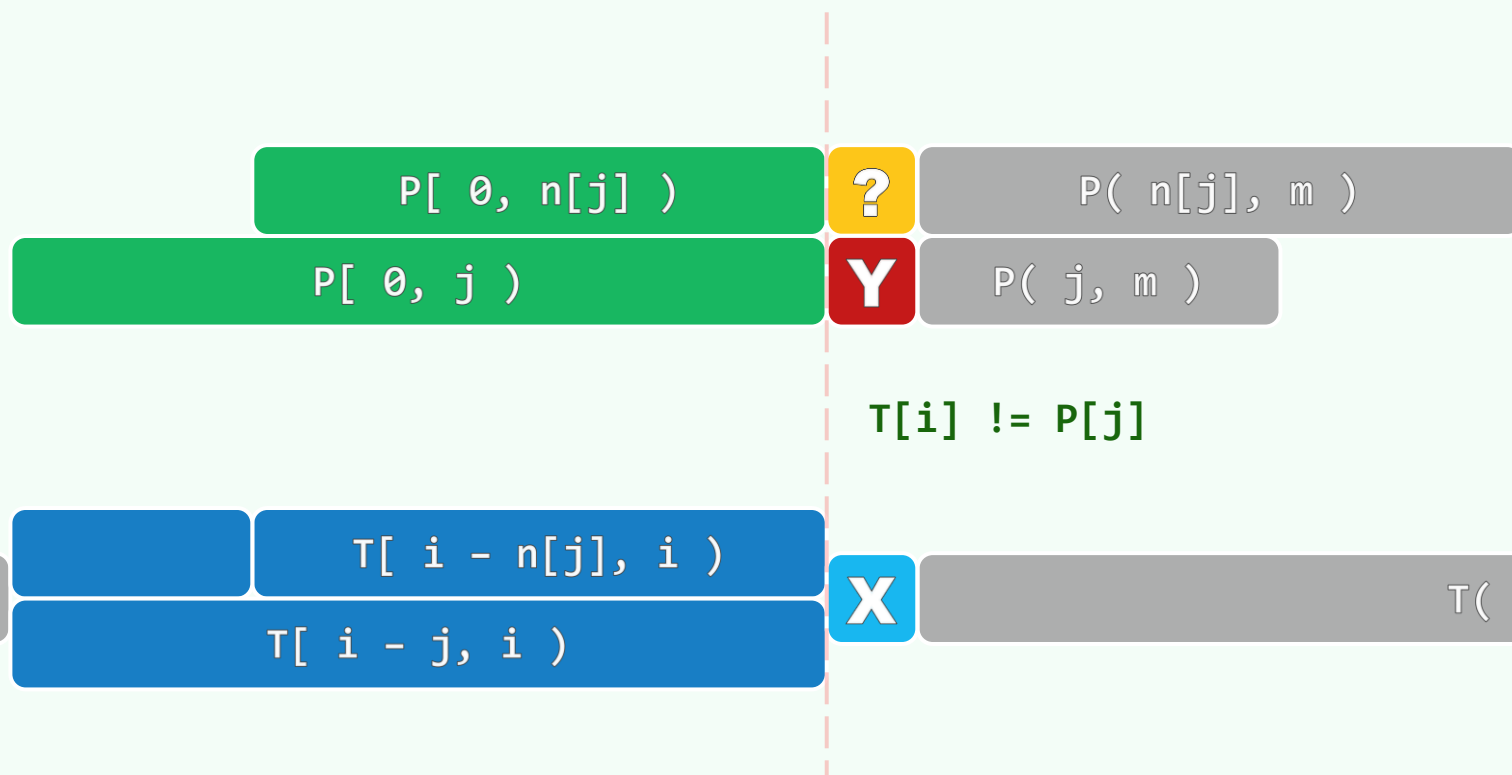
t: 不仅可以事先确定, 而且仅根据 $P[0, j) = T[i-j, i)$ 即可确定

❖ 视失败的位置j, 无非m种情况

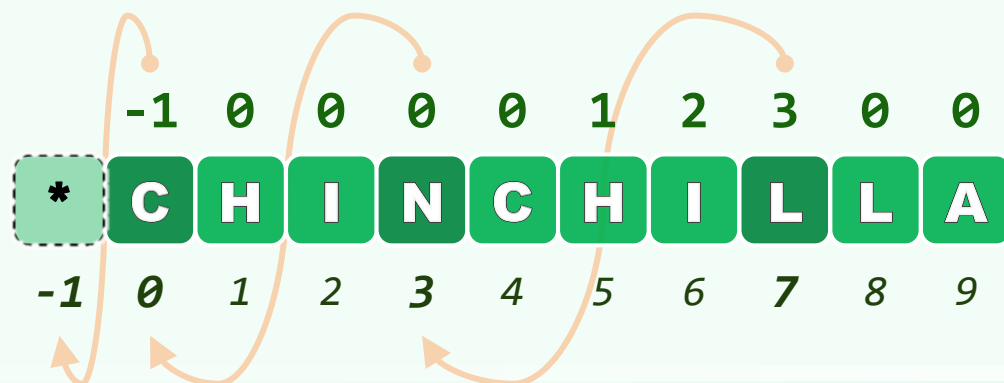
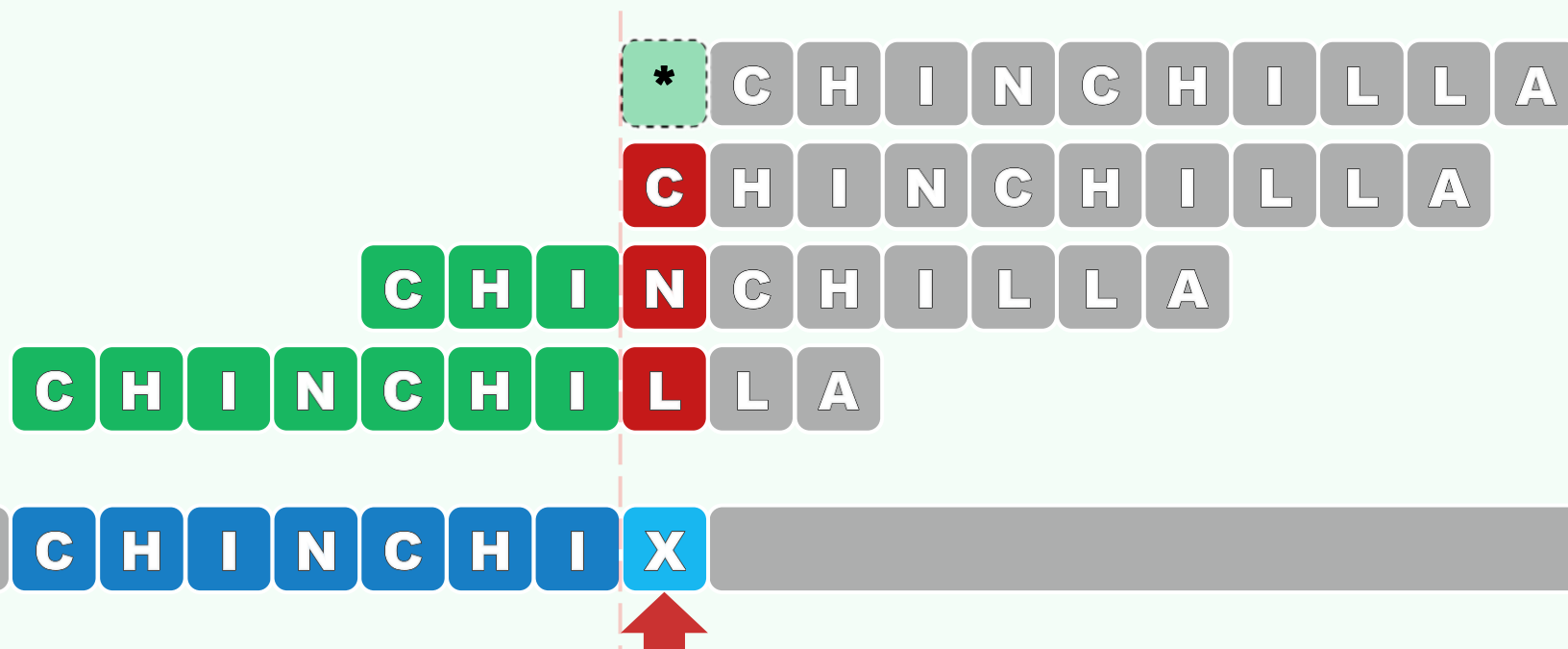
❖ 构造查询表 $next[0, m)$, 做好预案

❖ 一旦在 $P[j]$ 处失配, 只需

将j替换为 $next[j]$, 继续与 $T[i]$ 比对



实例



KMP算法

```
int match( char * P, char * T ) {  
    int * next = buildNext(P);  
    int n = (int) strlen(T), i = 0;  
    int m = (int) strlen(P), j = 0;  
    while ( (j < m) && (i <= n-m) )  
        if ( 0 > j || T[i] == P[j] ) {  
            i++; j++;  
        } else  
            j = next[j];  
    delete [] next;  
    return i - j;  
}
```



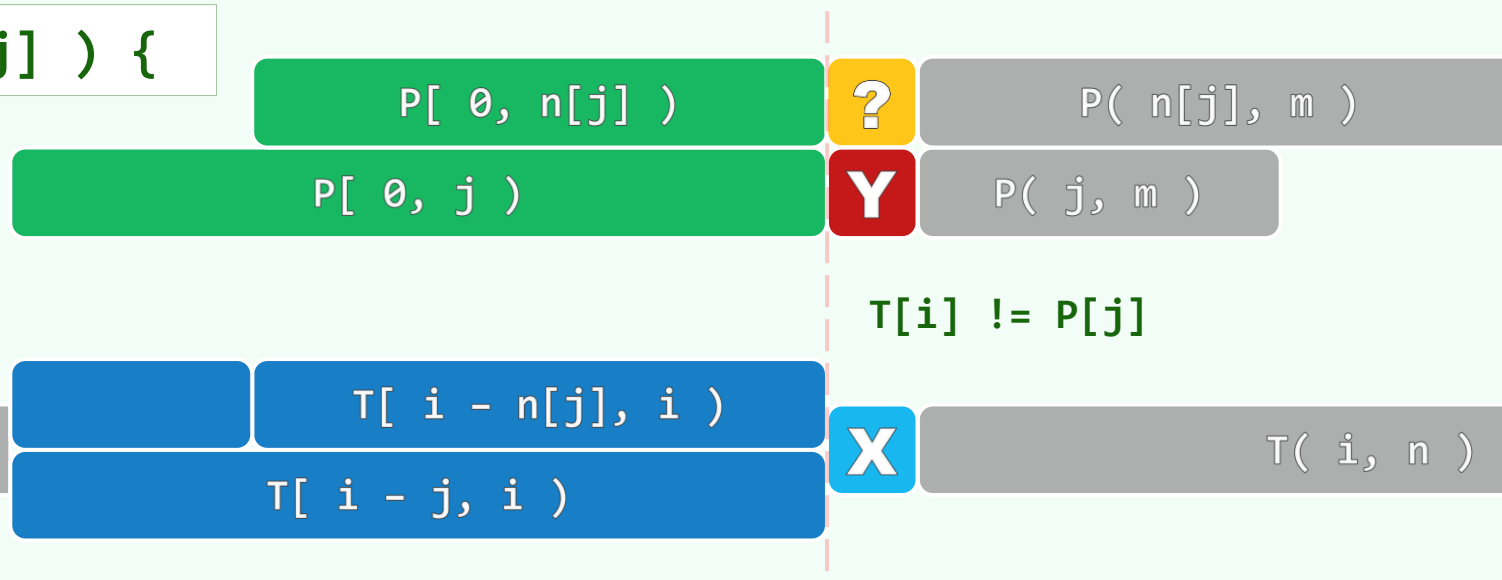
D. E. **K**nuth



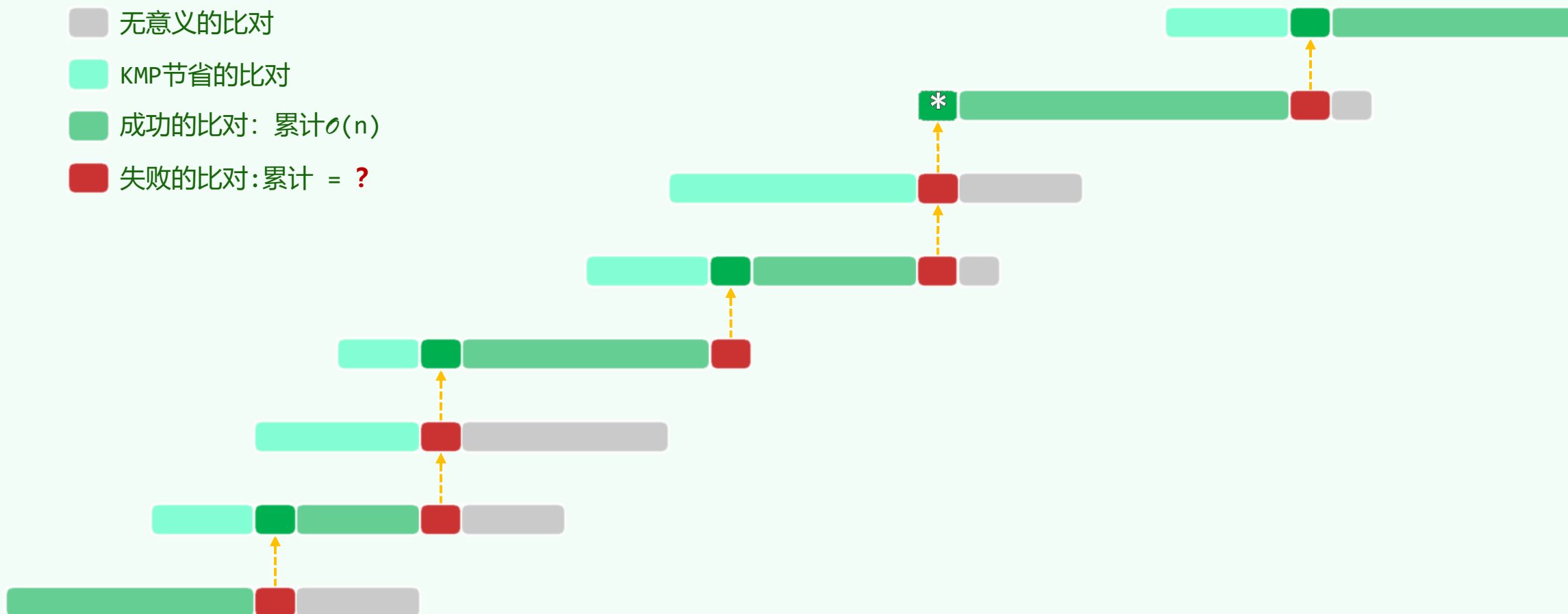
J. H. **M**orris



V. R. **P**ratt



快速右移 + 绝不回退



串

若想预见数学的未来，正确的方法是研究它的历史和现状

一切都是暂时的，转瞬即逝
而那逝去的将变为可爱

吳用再使時遷扮作伏路小軍，去曾頭市寨中，探聽他不出何意，所有陷坑，暗暗地記著，離寨多少路遠，總有幾處。時遷去了一日，都知備細，暗地使了記號，回報軍師

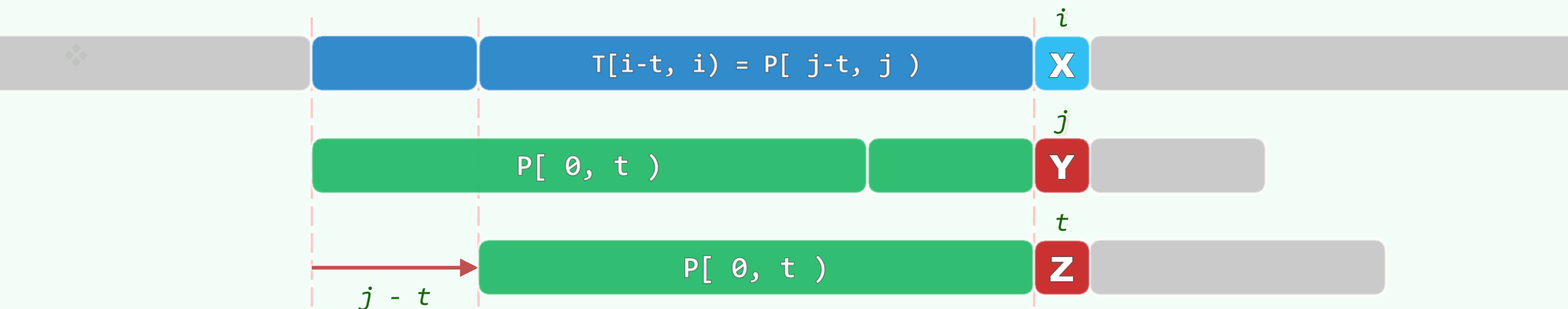
在这些胡思乱想里，我还怕我的记忆力不听我的使唤，怕记忆力出于疏忽而让我把同一件事写上两次。我讨厌在文章里再次认出自己，我炒冷饭向来是违心的

KMP算法：构造next表

邓俊辉

deng@tsinghua.edu.cn

最长自匹配：快速右移 + 绝不回退



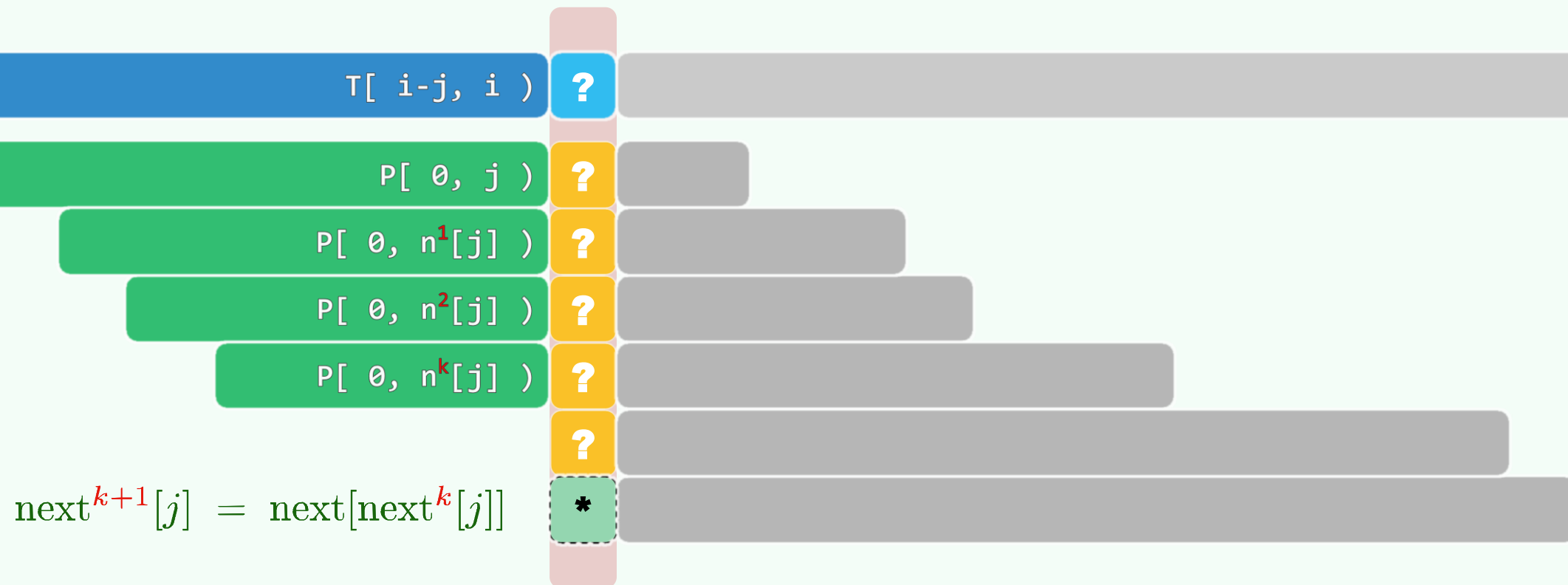
❖ $\forall 1 \leq j < m :$

$N(P, j) = \{ 0 \leq t < j \mid P[0, t) = P[j-t, j) \}$ //所有自匹配的长度

❖ $0 \in N(P, j) \neq \emptyset$ //必含0而非空，故可

❖ $next[j] = \max\{ N(P, j) \}$ //取最大长度：位移最小，不致回溯

自匹配：传递链



$$N(P, j) = \{ 0, \dots, next^3[j], next^2[j], next^1[j] \}$$

$$-1 \leftarrow 0 \leftarrow \dots \leftarrow next^3[j] \leftarrow next^2[j] \leftarrow next^1[j] \leftarrow next^0[j] = j$$

减而治之：由next[0]、next[1]、...、next[j]，如何得到next[j+1]？

-1 0 0 1 2 3 1 0
M A M A M M I A

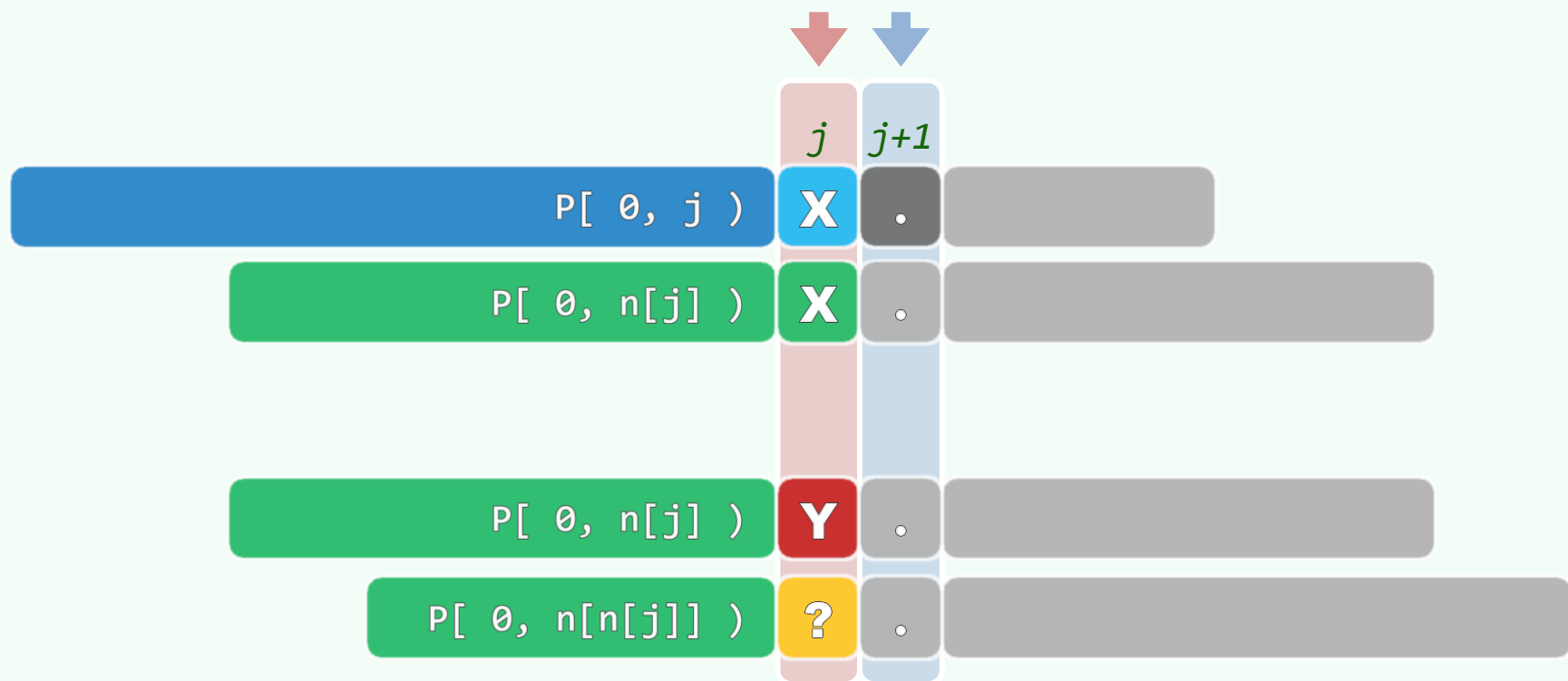
-1 0 0 1
M A M A M M I A
M A M A M M I A

-1 0 0 1 2
M A M A M M I A
M A M A M M I A

-1 0 0 1 2 3
M A M A M M I A
M A M A M M I A

-1 0 0 1 2 3 1
M A M A M M I A
M A M A M M I A
M A M A M M I A
M A M A M M I A

P在j+1处的自匹配，**至多**比在j处的自匹配多出一个字符...



$$\text{next}[j+1] = \text{next}[j] + 1 \quad \text{iff} \quad P[j] = P[\text{next}[j]]$$

串

KMP算法：均摊分析

13-C4

邓俊辉

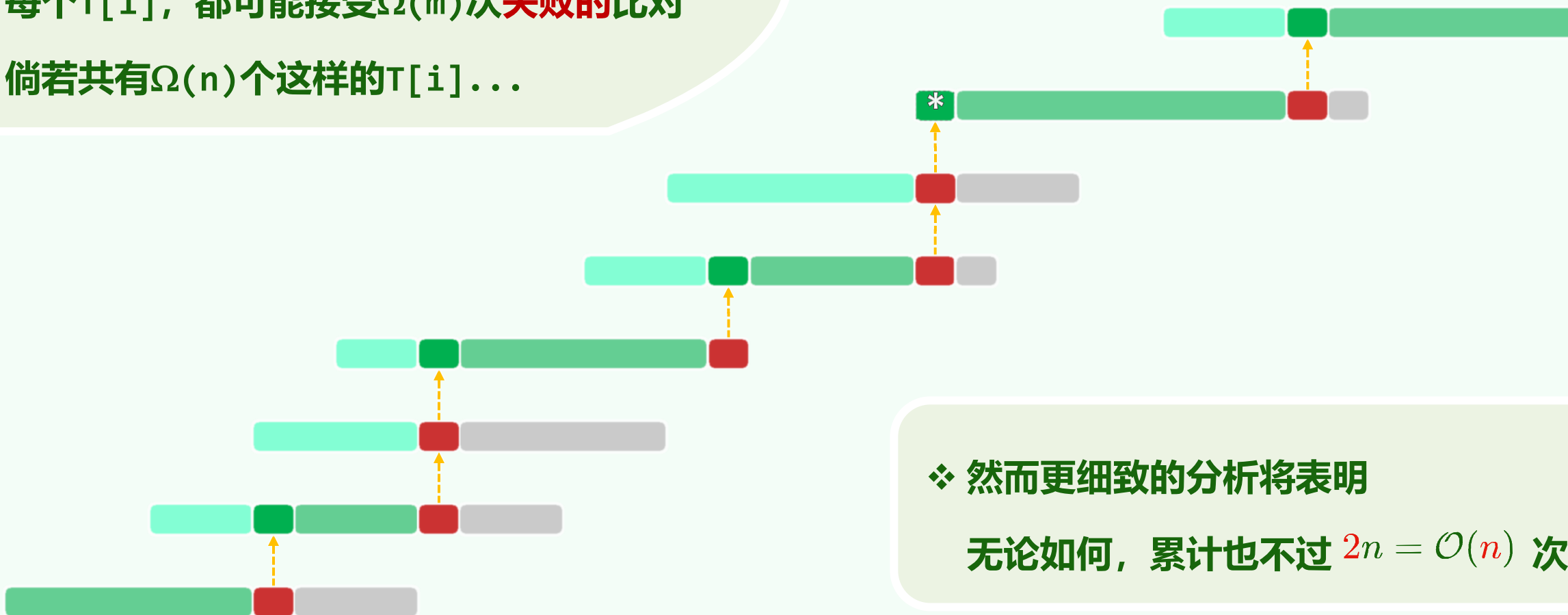
deng@tsinghua.edu.cn

失之東隅，收之桑榆

幸好过了一个冬天那女人又来了，两个人仍是逆时针绕着园子走，一长一短两个身影恰似钟表的两支指针

$\Omega(n*m)$?

❖ 每个 $T[i]$, 都可能接受 $\Omega(m)$ 次失败的比对
倘若共有 $\Omega(n)$ 个这样的 $T[i] \dots$



❖ 然而更细致的分析将表明

无论如何，累计也不过 $2n = \mathcal{O}(n)$ 次

$O(n)$ by Aggregate

❖ 令: $k = 2*i - j$

则: k 必随迭代而单调递增

❖ 故 k 也是迭代步数的上界

——相当于一个虽欠精准, 但还算够用的计步器

❖ 初始: $k = 0$

算法结束时: $k = 2*i - j \leq 2(n - 1) - (-1) = 2n - 1$

❖ while ((j < m) && (i < n))

if ($0 > j$ || $T[i] == P[j]$)

{ i ++; j ++; } // k 恰好加1

else

j = next[j]; // k 至少加1

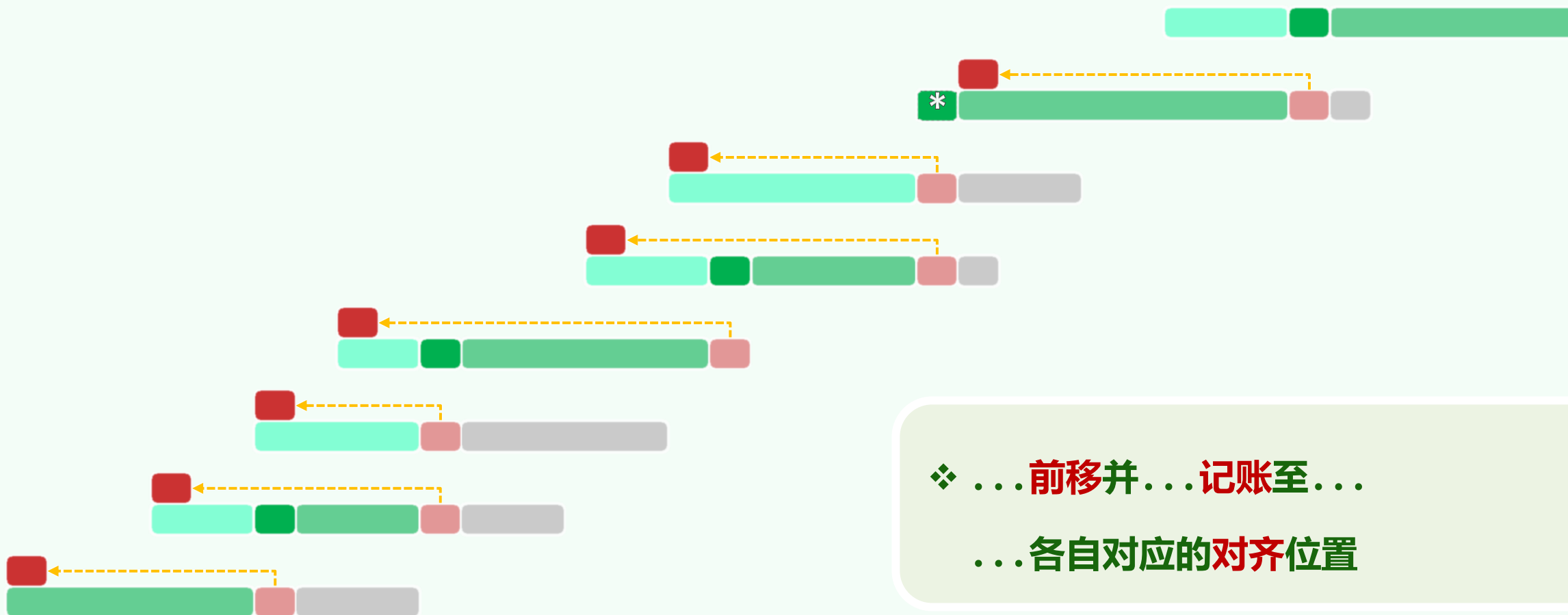
$O(n)$ by Accounting (1/2)

❖ 成功的比对：恰好覆盖T串，共计 $O(n)$ 次

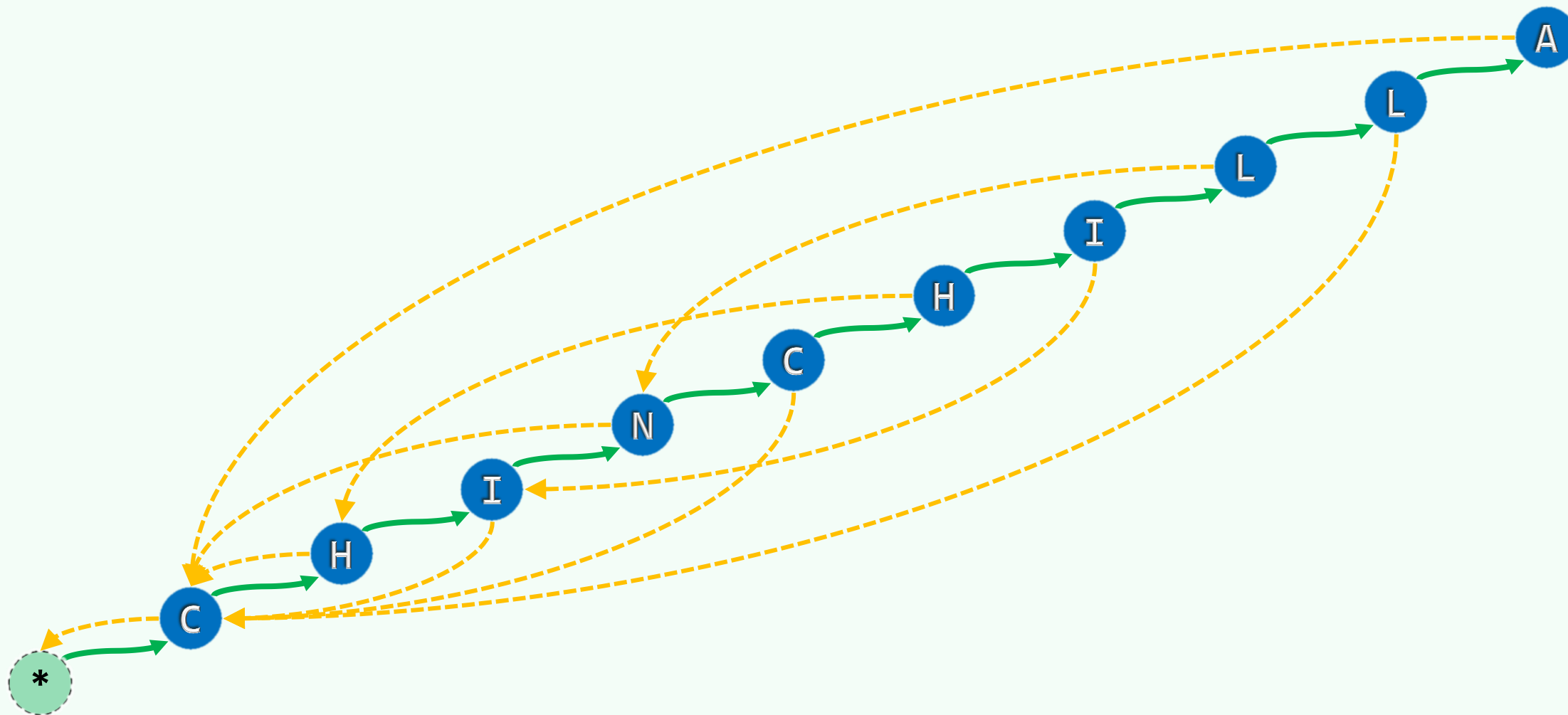


❖ 失败的比对：在同一位置i或多个
不能通过覆盖简明计数
但可以...

$O(n)$ by Accounting (2/2)



每一个P串，都是一台自动机



模式串 ~ 匹配算法

int match(char * T) { //对任一模式串 (比如P = **chinchilla**) , 可自动生成如下代码

int n = strlen(T); int i = -1; //文本串对齐位置

```
s_: ++i; // ↑
s0: (T[i] != 'C') ? goto s_ : if (n <= ++i) return -1; // [*] ~ ↑
s1: (T[i] != 'H') ? goto s0 : if (n <= ++i) return -1; // [*C] ~ [*]
s2: (T[i] != 'I') ? goto s0 : if (n <= ++i) return -1; // [*CH] ~ [*]
s3: (T[i] != 'N') ? goto s0 : if (n <= ++i) return -1; // [*CHI] ~ [*]
s4: (T[i] != 'C') ? goto s0 : if (n <= ++i) return -1; // [*CHIN] ~ [*]
s5: (T[i] != 'H') ? goto s1 : if (n <= ++i) return -1; // [*CHINC] ~ [*C]
s6: (T[i] != 'I') ? goto s2 : if (n <= ++i) return -1; // [*CHINCH] ~ [*CH]
s7: (T[i] != 'L') ? goto s3 : if (n <= ++i) return -1; // [*CHINCHI] ~ [*CHI]
s8: (T[i] != 'L') ? goto s0 : if (n <= ++i) return -1; // [*CHINCHIL] ~ [*]
s9: (T[i] != 'A') ? goto s0 : if (n <= ++i) return -1; // [*CHINCHILL] ~ [*]

return i - 10; // [*CHINCHILLA]
```

}

13-C5

串

KMP算法：再改进

有顏回者好學，不遷怒，不貳過

错误和挫折教训了我们，使我们比较地聪明起来了，我们的事情就办得好一些。任何政党，任何个人，错误总是难免的，我们要求犯得少一点。犯了错误则要求改正，改正得越迅速，越彻底，越好

邓俊辉

deng@tsinghua.edu.cn

反例

❖ 在 $T[3]$ 处

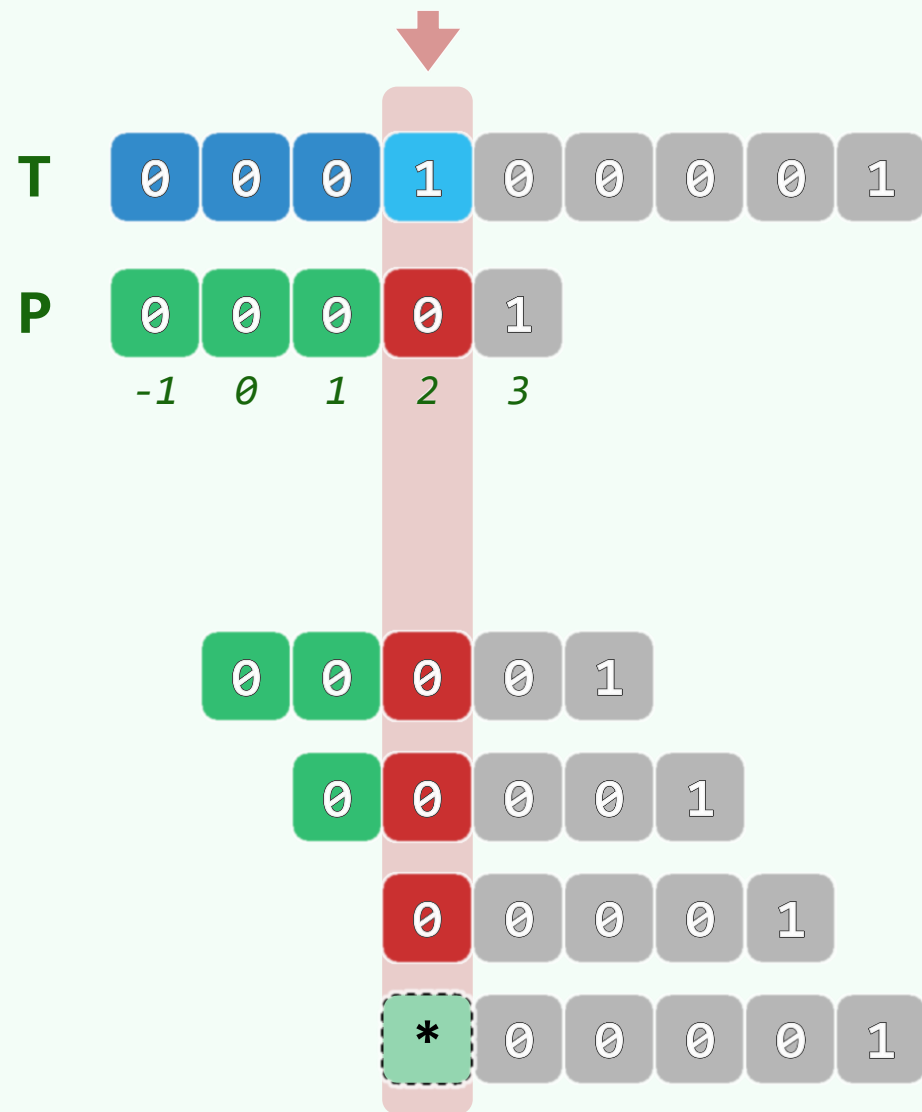
又：与 $P[3]$ 比对，失败

双：与 $P[2] = P[\text{next}[3]]$ 比对，失败

姦：与 $P[1] = P[\text{next}[2]]$ 比对，失败

姦：与 $P[0] = P[\text{next}[1]]$ 比对，失败

最终，才前进到 $T[4]$



根源

❖ 无需T串，即可在事先确定：

$P[3] =$

$P[2] =$

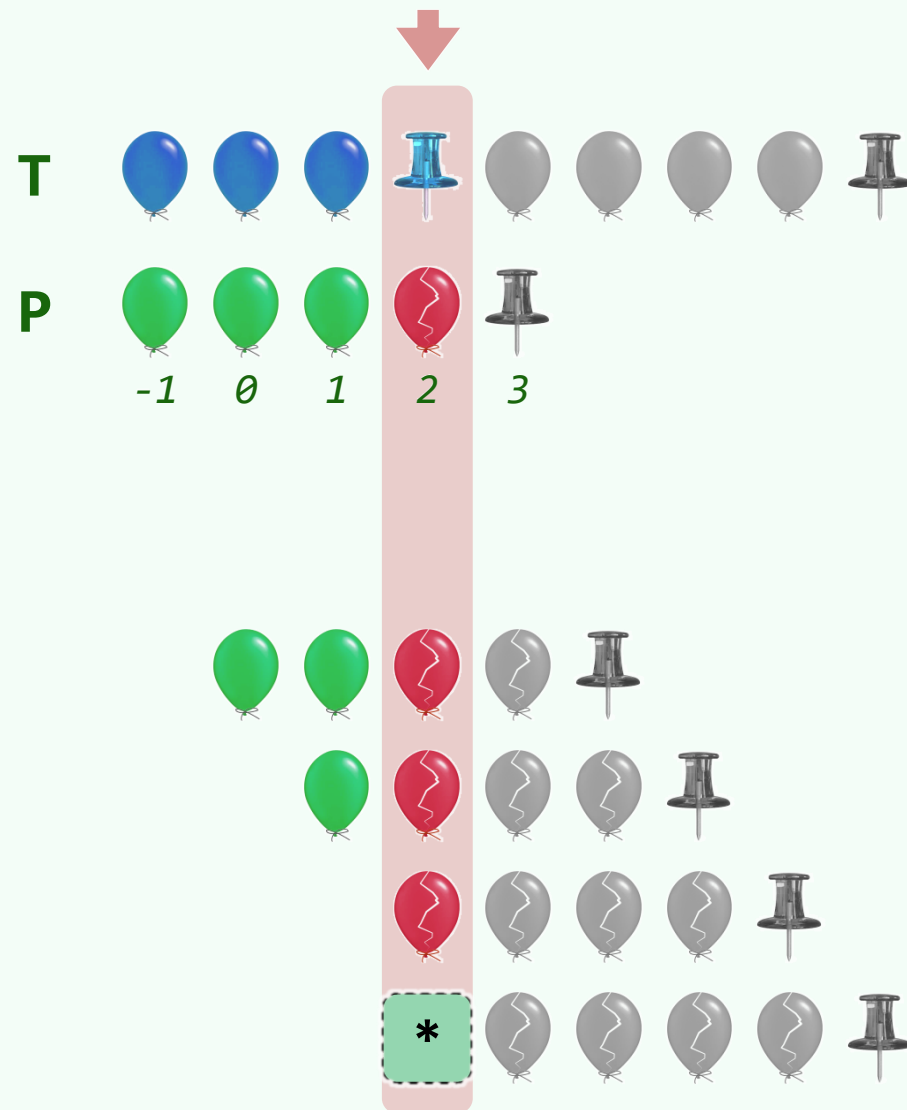
$P[1] =$

$P[0] = 0$

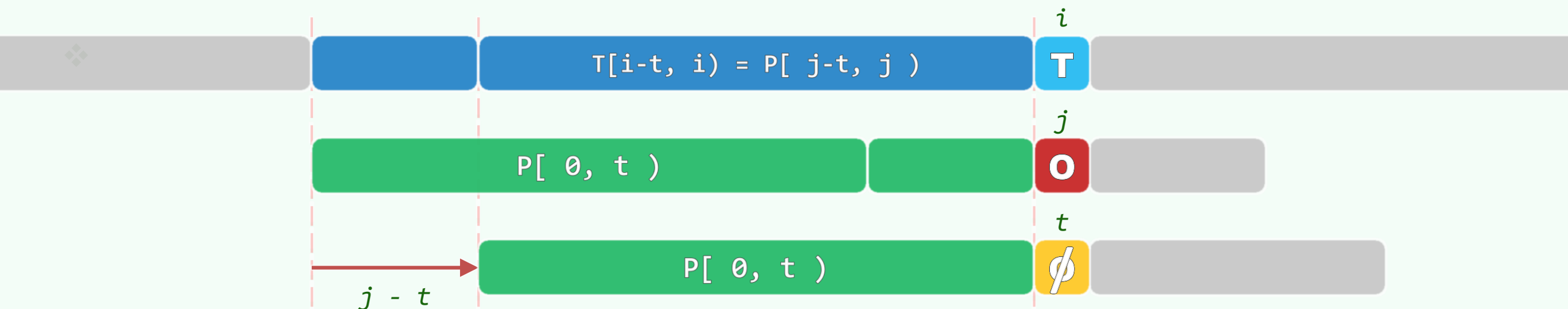
既然如此...

❖ 在发现 $T[3] \neq P[3]$ 之后，为何还要一错再错？

❖ 事实上，后三次比对本来都是可以避免的！



改进：经验 + 教训



❖ $\forall 0 \leq j < m :$

$$N(P, j) = \{ 0 \leq t < j \mid P[0, t) = P[j-t, j) \text{ and } P[t] \neq P[j] \} \cup \{ -1 \} \quad // \text{兼顾教训}$$

❖ $-1 \in N(P, j) \neq \emptyset$ // 必含-1而非空，故可

❖ $\text{next}[j] = \max\{ N(P, j) \}$ // 还是取最长

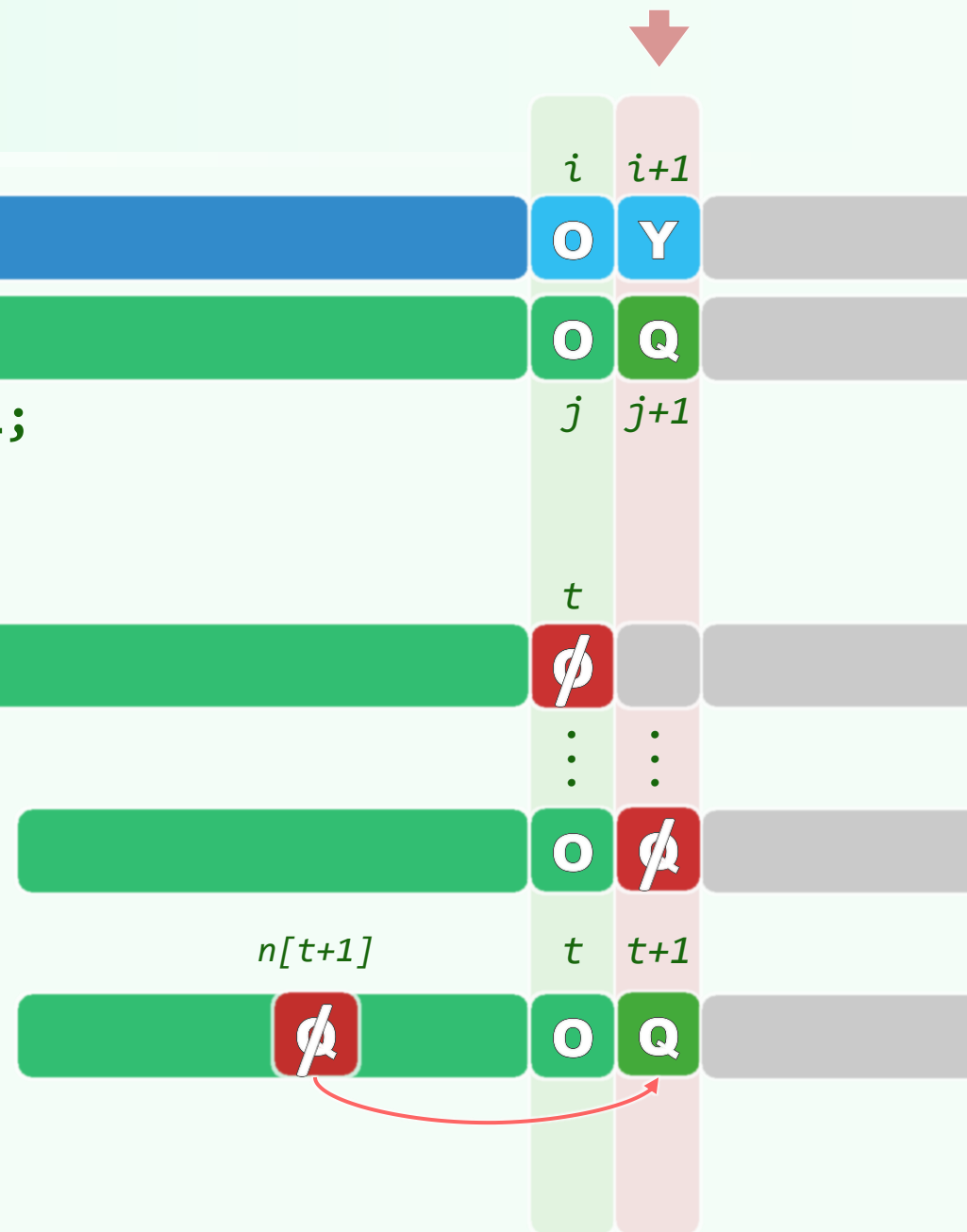
反观：就在刚刚，是如何确定next[j]的？

```
int* buildNext( char* P ) {  
    int m = strlen(P), j = 0;  
    int* next = new int[m]; int t = next[0] = -1;  
  
    while ( j < m - 1 )  
        if ( 0 <= t && P[t] != P[j] )  
            t = next[t];  
  
        else if ( P[++t] != P[++j] )  
            next[j] = t; //递增  
  
        else //P[next[t]] != P[t] == P[j]  
            next[j] = next[t]; //抄袭  
  
    return next;  
}
```



前进：那接下来，将如何确定 $\text{next}[j+1]$?

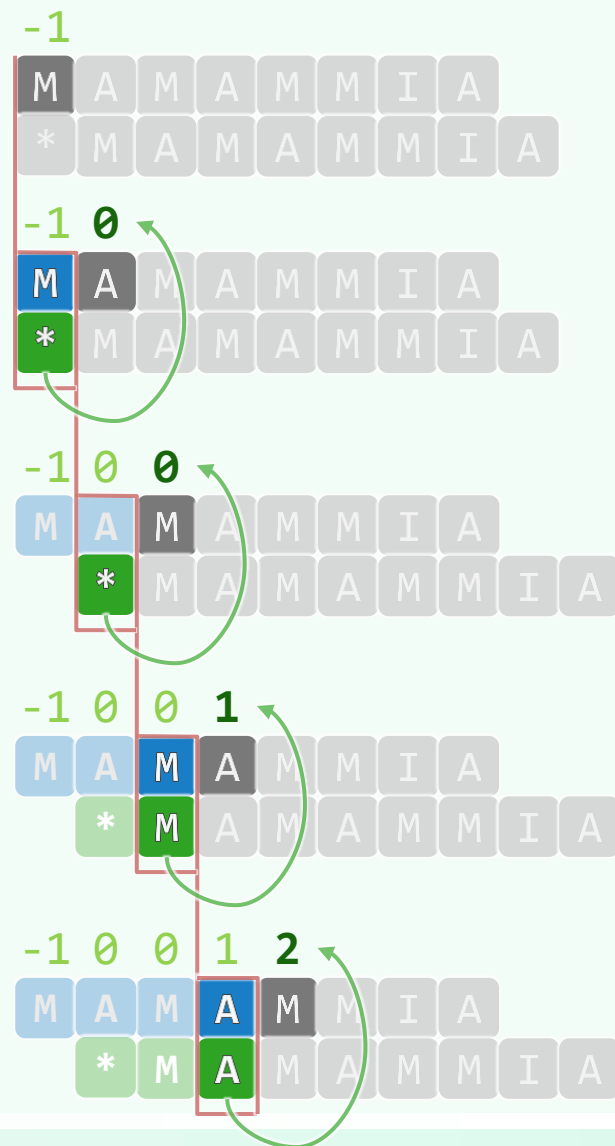
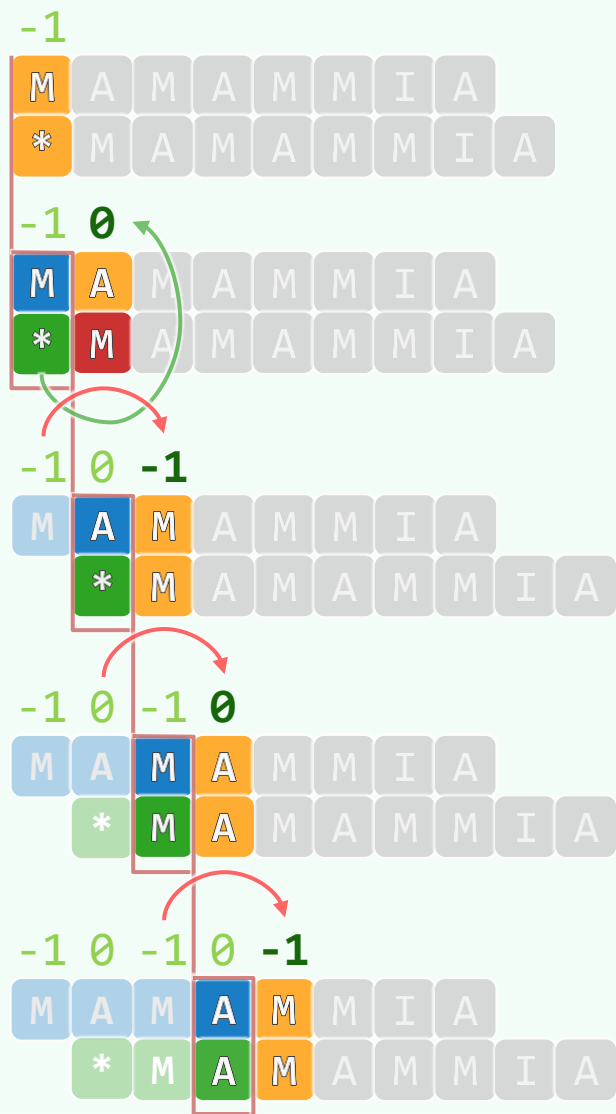
```
int* buildNext( char* P ) {  
    int m = strlen(P), j = 0;  
    int* next = new int[m]; int t = next[0] = -1;  
  
    while ( j < m - 1 )  
        if ( 0 <= t && P[t] != P[j] )  
            t = next[t];  
        else if ( P[++t] != P[++j] )  
            next[j] = t; //递增  
        else //P[next[t]] != P[t] == P[j]  
            next[j] = next[t]; //抄袭  
  
    return next;  
}
```



对比

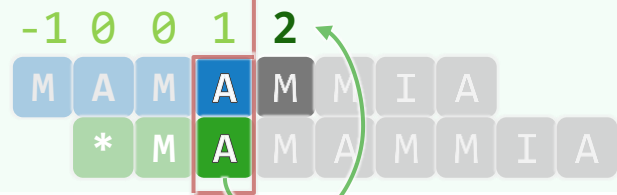
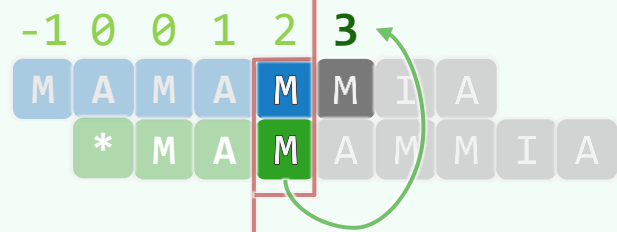
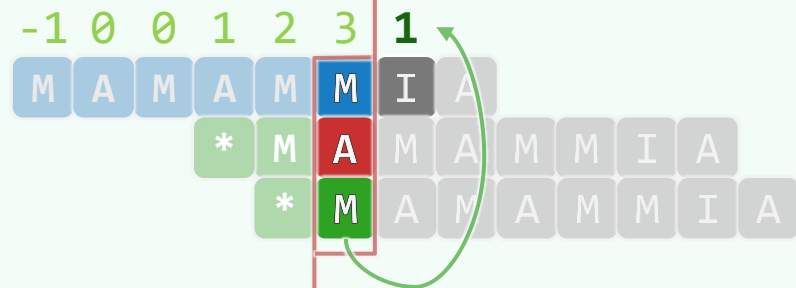
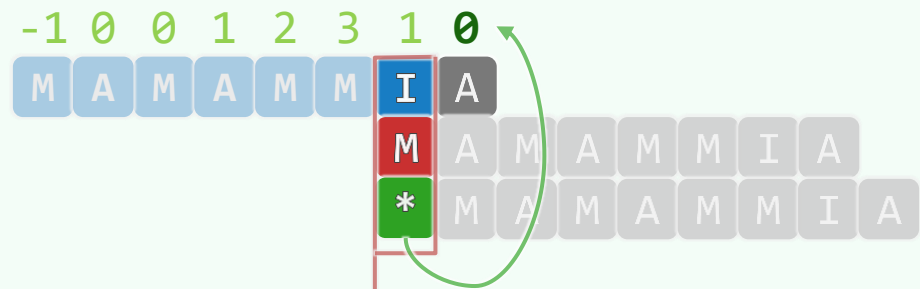
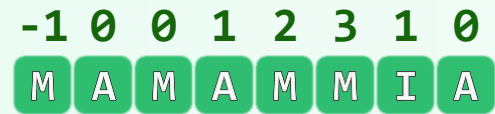
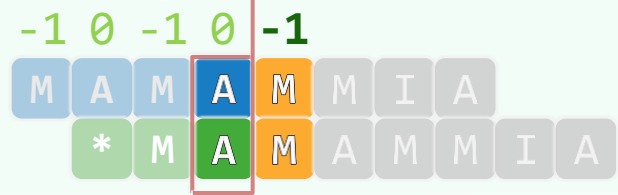
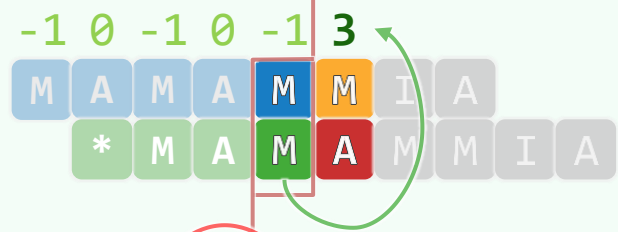
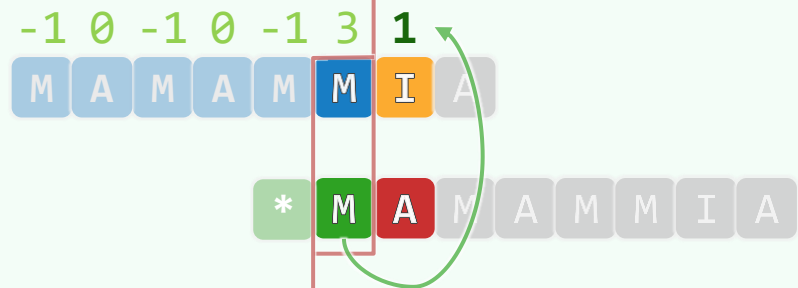
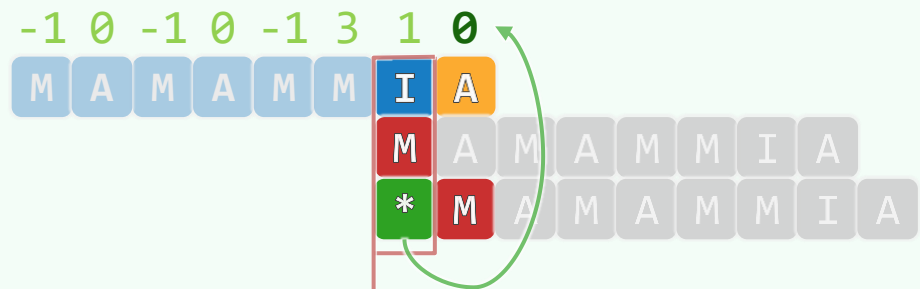
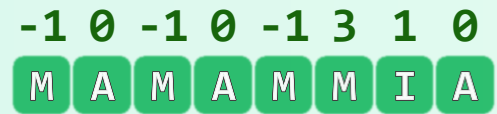
-1 0 -1 0 -1 3 1 0
M A M A M M I A

-1 0 0 1 2 3 1 0
M A M A M M I A



对比

-1	0	-1	0	-1	3	1	0
M	A	M	A	M	M	I	A



小结

❖ 充分利用**以往的**比对所提供的信息

模式串快速右移，文本串无需回退

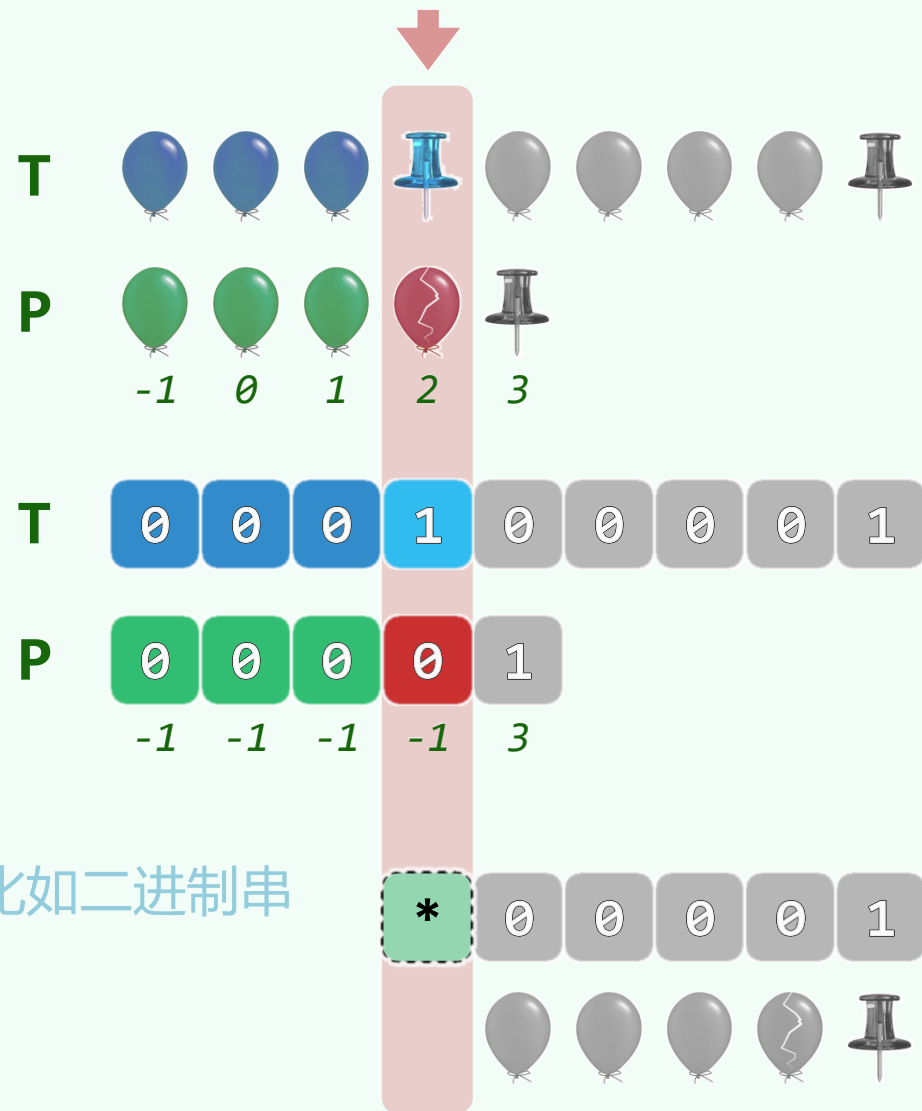
❖ **经验** ~ 以往**成功**的比对: $T[i-j, i)$ **是什么**

教训 ~ 以往**失败**的比对: $T[i]$ **不是什么**

❖ 特别适用于**顺序**存储介质

❖ 单次匹配概率越**大**（字符集越**小**），优势越**明显** //比如二进制串

否则，与蛮力算法的性能相差无几...



串

BM算法：BC策略：以终为始

Bottom Line On Top

我劝你还是努力去利用那过去之石，来磨砺这现在之刀吧。因为生活本身就有许多失败，你多得着一次，不就是减少了一次吗？！

一般地说来，与其失之过迟，不如失之过早。因为后者的损失较之前者为小，而其利益，则是有备无患，根本上立于不败之地

邓俊辉

deng@tsinghua.edu.cn

尽早试错，善待教训

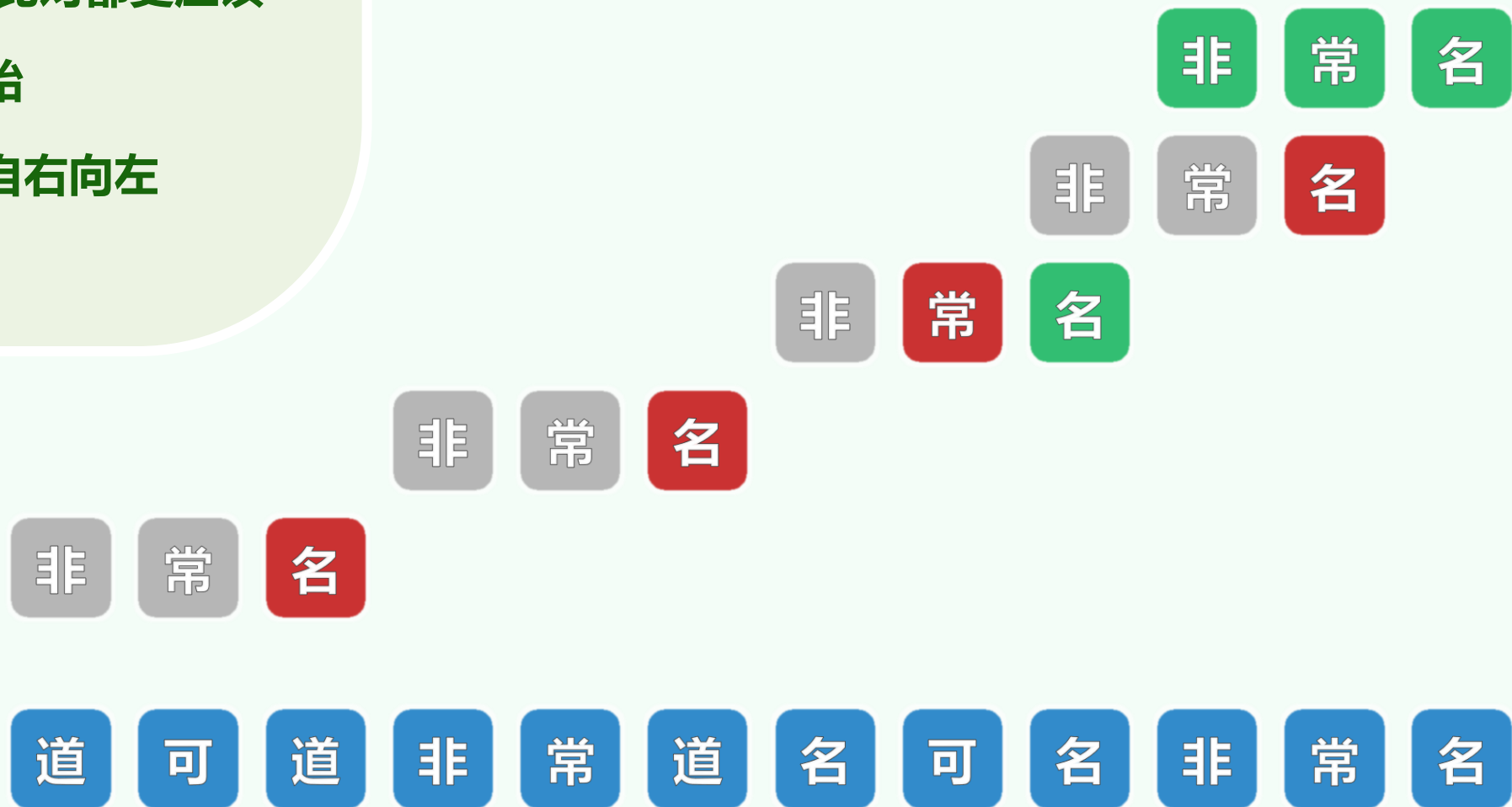


以终为始，大事优先

❖ 既如此，每一趟比对都更应该

- 从**末**字符开始
- 自后向前，自右向左

❖ $4 + 4 < 12$



[Boyer + Moore, 1977] A Fast String Searching Algorithm

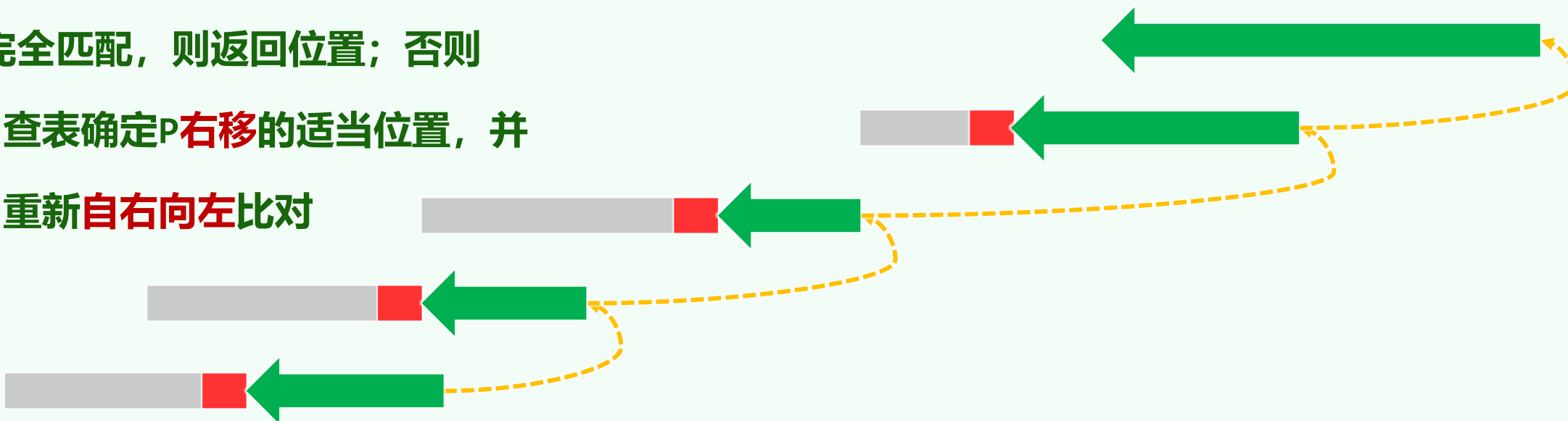
❖ 预处理：根据模式串P，预先构造bc[]表（以及gs[]表）

与next[]类似，为日后的局部适配提供快捷处置的建议

❖ 迭代：自右向左依次比对字符，找到极大的匹配后缀

若完全匹配，则返回位置；否则

- 查表确定P右移的适当位置，并
- 重新自右向左比对



串

BM算法：BC策略：坏字符

13-D2

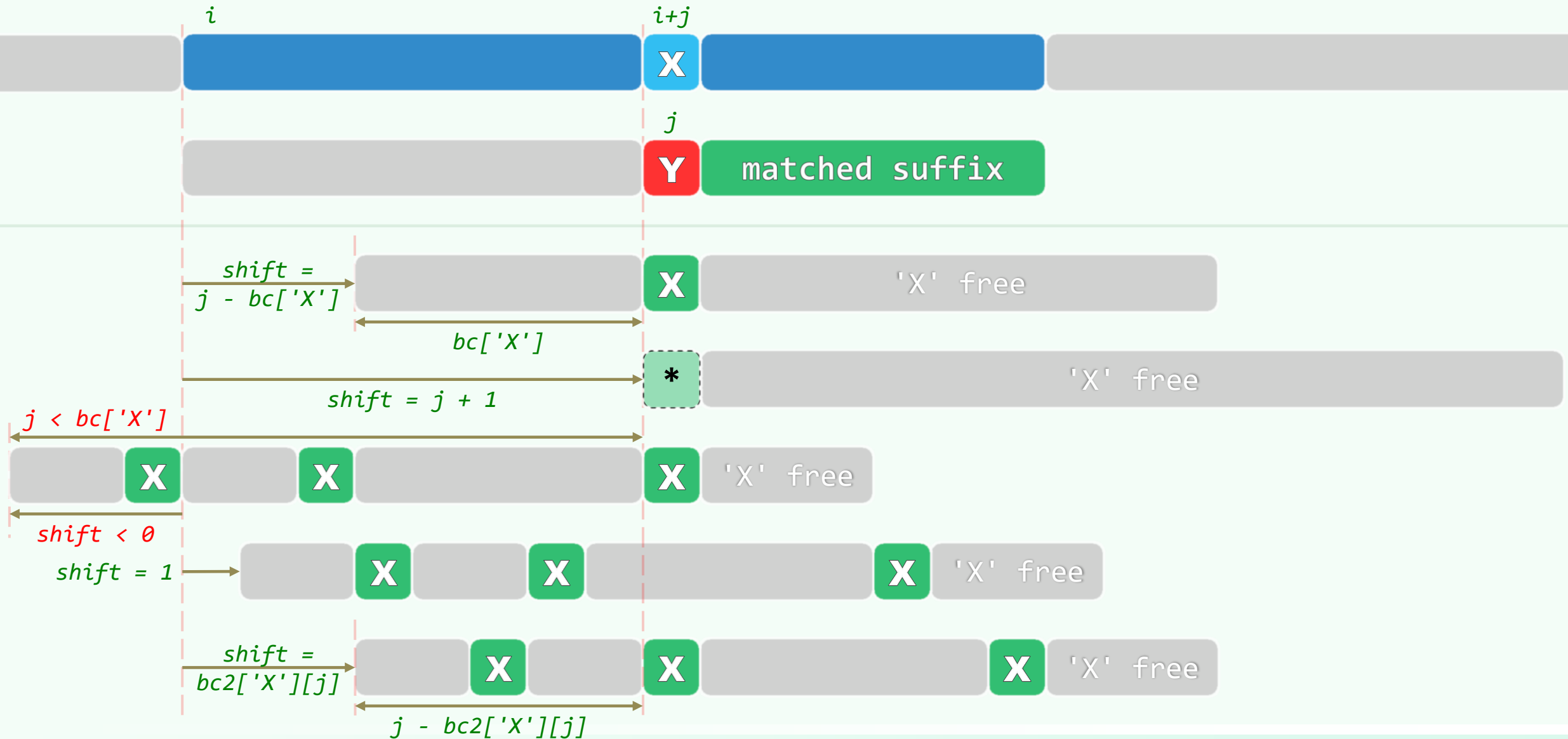
邓俊辉

deng@tsinghua.edu.cn

君子有諸己而後求諸人，無諸己而後非諸人

子路，人告之以有過則喜；禹，聞善言則拜

Bad-Character Shift



实现

```
int match( char* P, char* T ) { //BM算法: 简化版, 只考虑Bad Character Shift

    int* bc = buildBC( P ); //预处理

    int n = strlen( T ), i = 0;

    int m = strlen( P ), j = m-1;

    while ( (0 <= j) && (i <= n-m) ) //自右向左逐个比对字符

        if (T[i+j] == P[j]) j--; //若匹配, 则转到下一对字符

        else { i += max(1, j - bc[T[i+j]]); j = m-1; } //否则, 借助BC表快速右移模式串

    delete [] bc; return i; //返回最终的对齐位置

} //match
```

串

BM算法: BC策略: 构造bc[]

13-D3

I have learned more from my failures ... failures are much more
fun to hear about afterwards; they are not so funny at the time.

好吧, 你这个倔强的孩子, 那条路很难走, 一路小心!

邓俊辉

deng@tsinghua.edu.cn

画家算法

```
int * buildBC( char * P ) {
```

M	A	M	M	A	M	I	A
0	1	2	3	4	5	6	7

```
    int * bc = new int[ 256 ];
```

```
    for ( size_t j = 0; j < 256; j++ ) bc[j] = -1;
```

```
    for ( size_t m = strlen(P), j = 0; j < m; j++ )
```

```
        bc[ P[ j ] ] = j; //painter's algorithm
```

```
    return bc;
```

```
    } //O(s+m)
```

return bc;												5														
7	} // O(s+m)											3														
4												2														
1	^	^	^	^	^	^	^	6	^	^	^	0	^	^	^	^	^	^	^	^	^	^	^	^	^	^
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	

串

BM算法：BC策略：性能分析

13-D4

邓俊辉

deng@tsinghua.edu.cn

要不犯十四次，甚至一百四十次错误，就不会得到任何一个真理

最好情况

❖ $\mathcal{O}(n/m)$ —— 除法？没错！比如：

$$T = \begin{array}{|c|c|c|c|c|} \hline x & x & x & x & 1 \\ \hline \end{array} \begin{array}{|c|c|c|c|c|} \hline x & x & x & x & 1 \\ \hline \end{array} \begin{array}{|c|c|c|c|c|} \hline x & x & x & x & 1 \\ \hline \end{array} \begin{array}{|c|c|c|c|c|} \hline x & x & x & x & 1 \\ \hline \end{array} \begin{array}{|c|c|c|c|c|} \hline x & x & x & x & 1 \\ \hline \end{array}$$
$$P = \begin{array}{|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 \\ \hline \end{array}$$

❖ 一般地： 只要P不含T[i+j]，即可直接移动m个字符

仅需单次比较，即可排除m个对齐位置

❖ 单次匹配概率越小，性能优势越明显 //大字母表：ASCII、Unicode

❖ P越长，这类移动的效果越明显

最差情况

❖ $O(n \times m)$ —— 退化为蛮力算法？是的！比如：

$T =$

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

$P =$

1	0	0	0	0
---	---	---	---	---

❖ 每轮迭代，都要在扫过**整个** P 之后，方能确定右移**一个**字符

此时，须经 **m 次**比较，方能排除**单个**对齐位置

❖ 单次匹配概率**越大**的场合，性能**越接近于**蛮力算法 //小字母表Bitmap + DNA

❖ 反思：借助以上 $bc[]$ 表，仅仅利用了**失配**比对提供的信息（**教训**）！

类比：可否仿照KMP，同时利用起**匹配**比对提供的信息（**经验**）？

串

BM算法：GS策略：好后缀

13-E1

是谁出的题这么的难，到处全都是正确答案

如果说即使是它们尚未为人所知，他们也可以根据过去的经验，以很大的概率预见未来的事件，那么为什么以某种或然性，根据人类历史的结果来追踪人类未来命运的史表，就应该被看成是一桩虚无缥缈的事业呢？

邓俊辉

deng@tsinghua.edu.cn

Good Suffix = 匹配的后缀 = 经验

❖ 首趟比对虽失败，却积累了足够的经验

(匹配的后綴 **ATCH**) // 好后綴

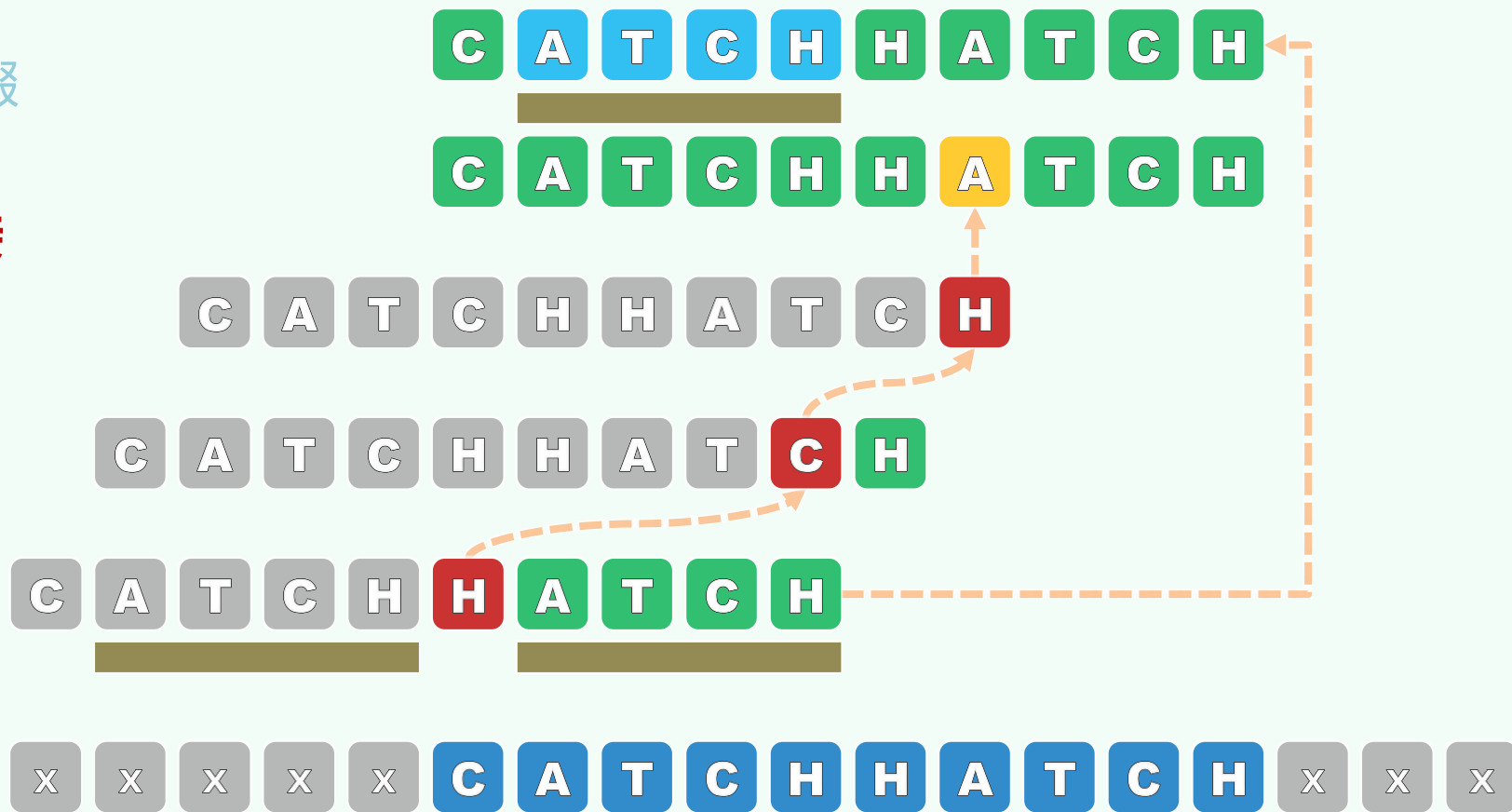
❖ 据此，可省去中间两趟，而直接

转至最后一趟 (P右移5个字符)

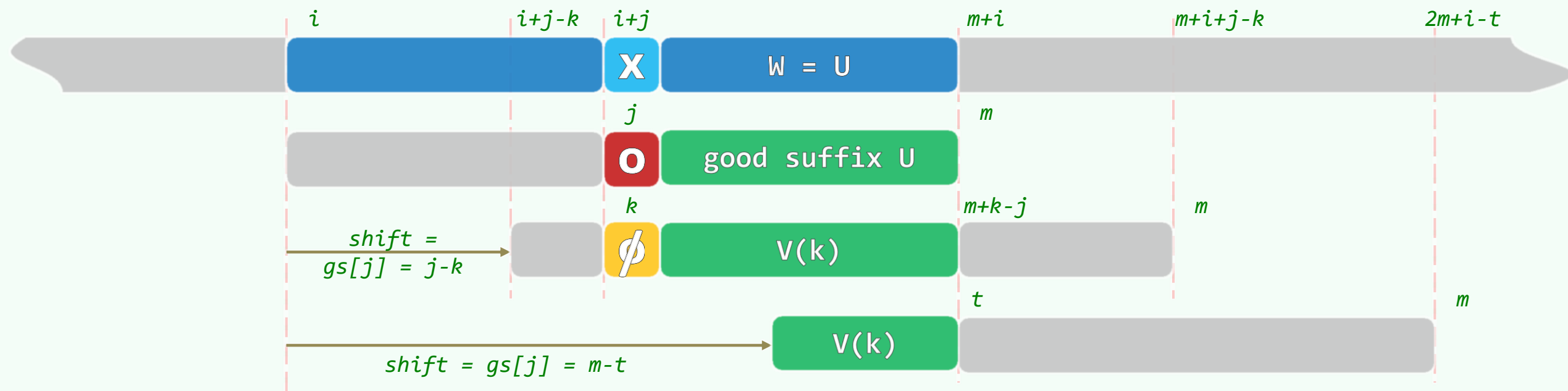
❖ 这一规律与技巧与KMP

如出一辙，只不过

前后颠倒而已...



GS Shift: 构思



❖ 若比对中断于 $T[i+j] = \boxed{X} \neq \boxed{O} = P[j]$

($U = P(j, m)$ 即为好后缀)

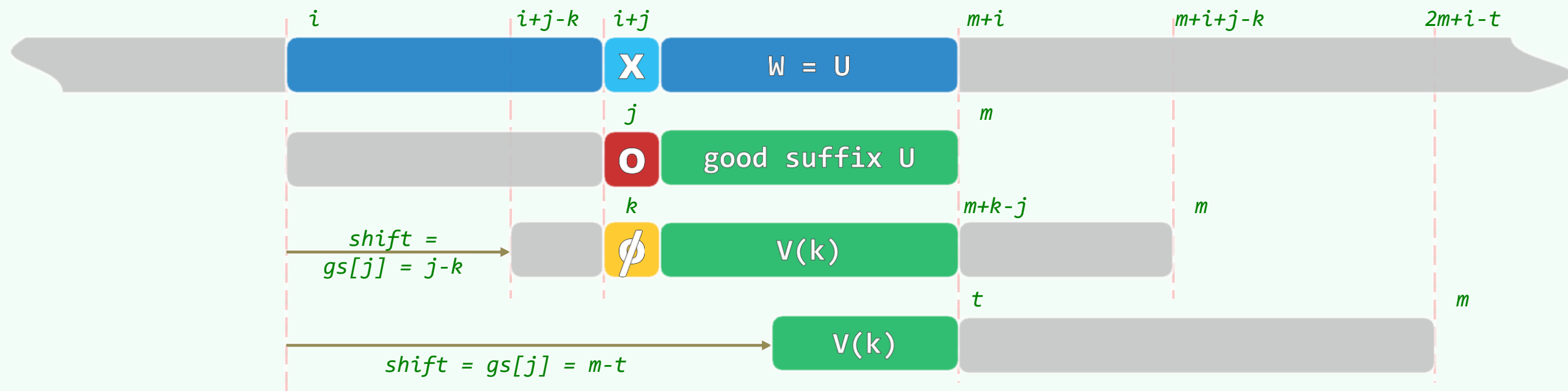
则下一对齐位置, 取决于 $P[j]$ 的替代者 $P[k]$

❖ k 最好能够

- 经验保持: U 与 $V(k) = P(k, m+k-j)$ 匹配

- 教训汲取: $P[k] \neq \boxed{O} = P[j]$

GS Shift: 算法



- ❖ 完全匹配：在这样的 k 中，取**最大（最靠后）**者 //移动距离**最小**
- ❖ 部分匹配：若**没有**这样的 k ，则在所有前缀 $P[0, t)$ 中，取与 **U** 匹配的**最长者** //有可能 $t=0$
- ❖ 无论如何，位移量仅取决于 **j** 和 **P** 本身——亦可**预先计算**，并**制表待查**

实例

8 8 8 8 8 4 8 1
非 曰 静 也 善 故 静 也

圣 人 之 静 也 非 曰 静 也 善 故 静 也

非 曰 静 也 善 故 静 也

+1
非 曰 静 也 善 故 静 也

+4
非 曰 静 也 善 故 静 也

串

BM算法: GS策略: 构造gs表

13-E2

邓俊辉

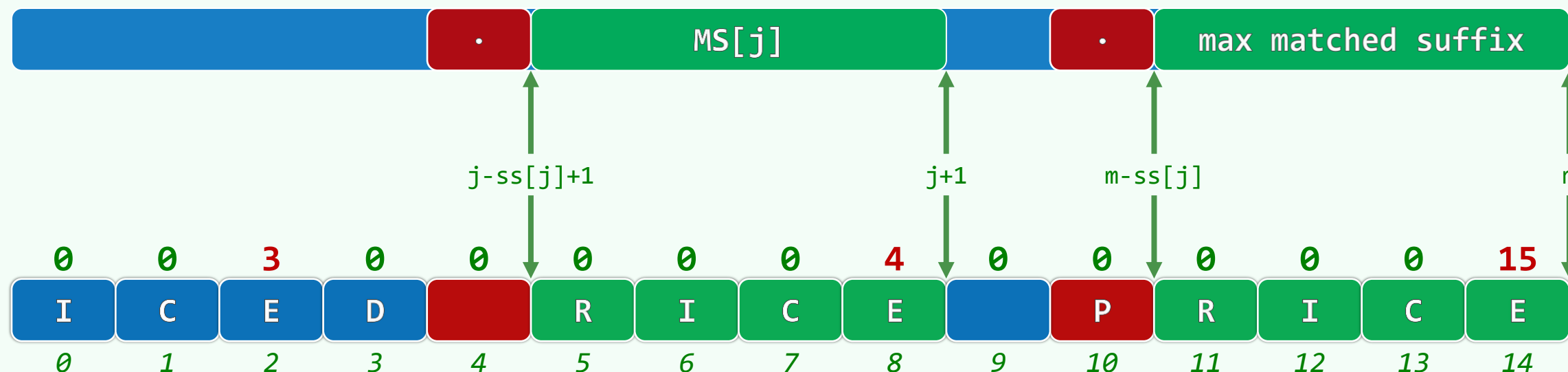
deng@tsinghua.edu.cn

Wrong cannot afford defeat, but Right can.

MS[] $\rightarrow$ ss[]

❖ 对任一 $0 \leq j < m$, 令: $ss[j] = \max\{ 0 \leq s \leq j + 1 \mid P(j - s, j] = P[m - s, m) \}$

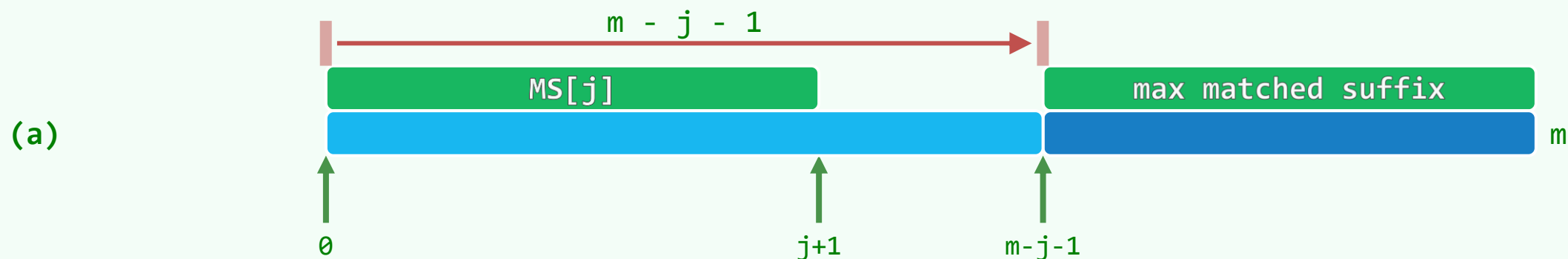
❖ 于是, $MS[j] = P(j - ss[j], j]$ 就是 $P[0, j]$ 所有后缀中, 与P的某一后缀匹配的最长者



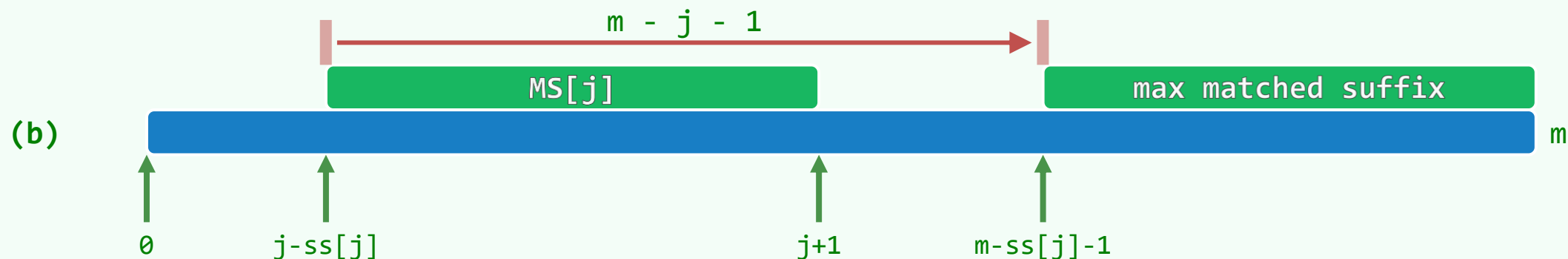
❖ 实际上, $ss[0, m)$ 表中蕴含了 $gs[0, m)$ 表的所有信息 //无非两种情况...

$ss[] \rightarrow gs[]$

a) 若 $ss[j] = j + 1$, 则对于任何 $i < m - j - 1$, $m - j - 1$ 必是 $gs[i]$ 的一个候选

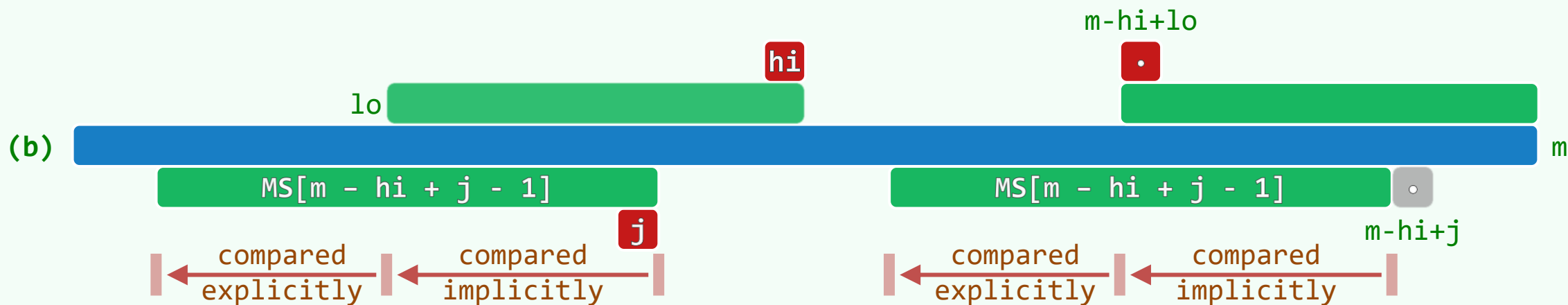
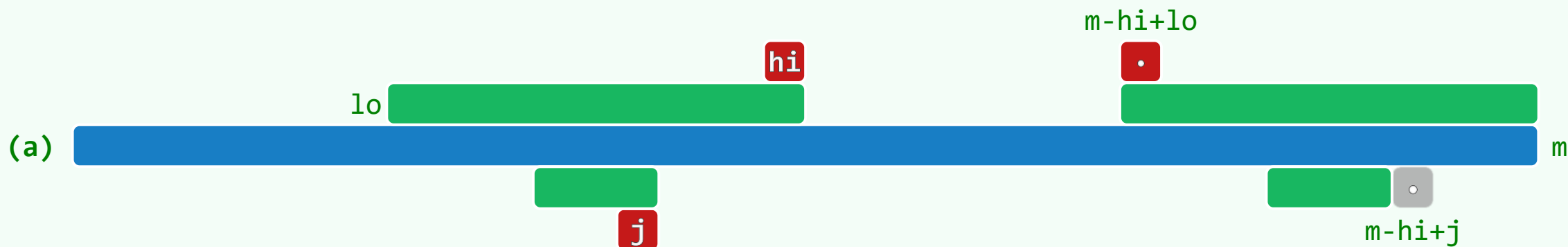


b) 若 $ss[j] \leq j$, 则 $m - j - 1$ 必是 $gs[m - ss[j] - 1]$ 的一个候选



构造ss[]

❖ 蛮力地对每个字符都扫描一趟，累计 $\mathcal{O}(m^2)$ ；自后向前**逆向**扫描，只需 $\mathcal{O}(m)$ 时间 //习题[11-6]



串

BM算法: GS策略: 综合性能

13-E3

邓俊辉

deng@tsinghua.edu.cn

叹人生，不如意事，十常八九

婚姻是否能取得最大的幸福，在很多方面要取决于男女双方是不是相配；
不过，要想在各个方面都相配的话，那是十分愚蠢的

性能

❖ 空间 = $|bc| + |gs| = O(|\Sigma| + m)$

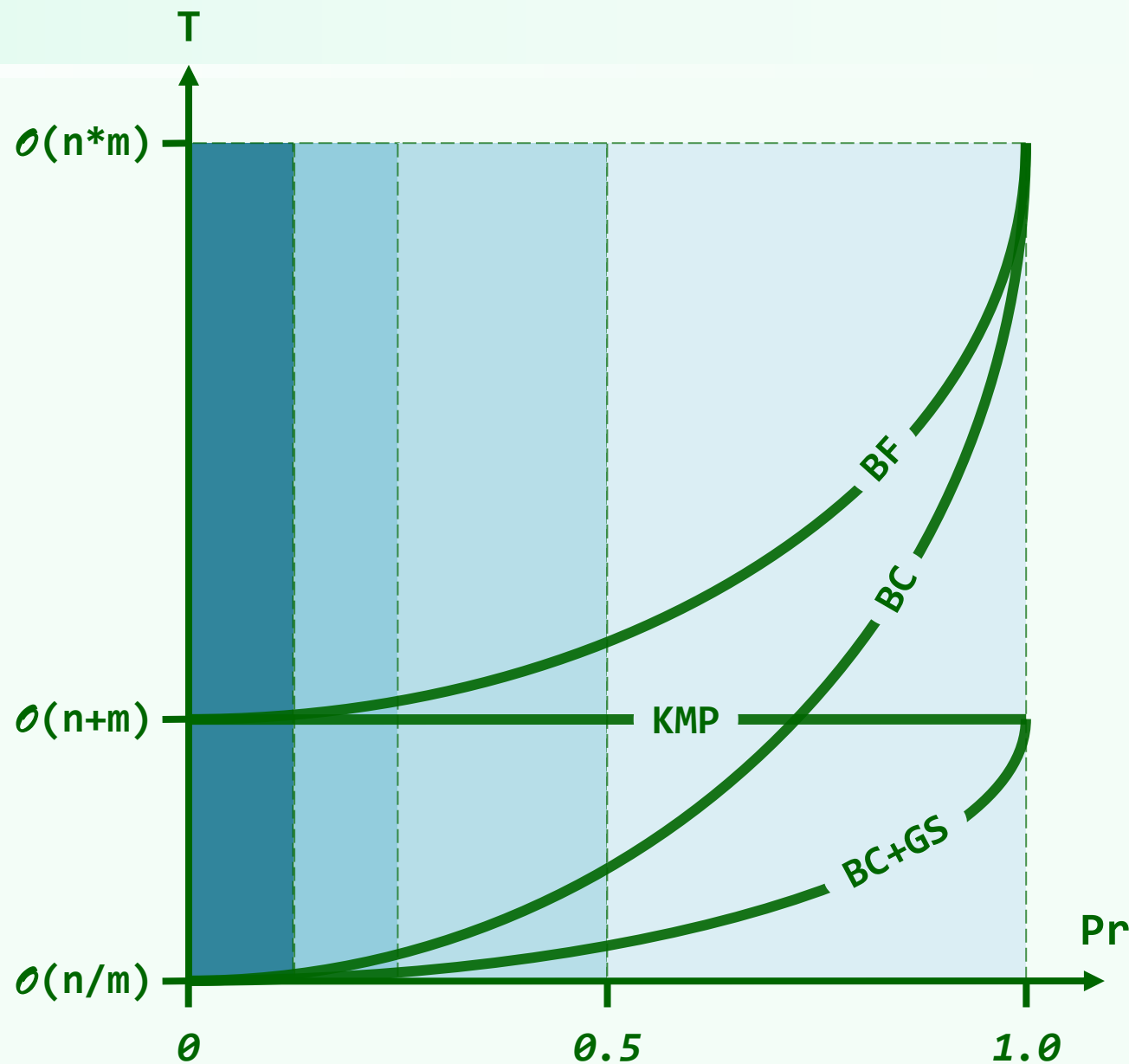
❖ 预处理: $O(|\Sigma| + m)$

❖ 查找效率

- 最好 $O(n / m)$
- 最差 $O(n + m)$ //分析方法类似KMP

❖ 关键因素

- 单次比对成功的概率
- 通常, $Pr = 1/s$



串

Karp-Rabin算法：串即是数

13-F1

All things are number. - Pythagoras

Every entity is like a living mirror, containing an
image of the universe, in its own way. - Leibniz

邓俊辉

deng@tsinghua.edu.cn

凡物皆数：Gödel Numbering

- ❖ 逻辑系统的符号、表达式、公式、命题、定理、公理等，均可表示为**自然数**
- ❖ 每个有限维的自然数**向量**（包括字符串），都唯一对应于某个**自然数**
- ❖ 素数序列： $p(k)$ = 第 k 个素数 = 2, 3, 5, 7, 11, 13, 17, 19, ...

$$\langle a_1, a_2, \dots, a_n \rangle \Rightarrow p(1)^{1+a_1} \times p(2)^{1+a_2} \times \dots \times p(n)^{1+a_n}$$

$$\langle \boxed{3} \quad \boxed{1} \quad \boxed{4} \quad \boxed{1} \quad \boxed{5} \quad \boxed{9} \quad \boxed{2} \quad \boxed{6} \rangle$$
$$2^{\boxed{4}} \times 3^{\boxed{2}} \times 5^{\boxed{5}} \times 7^{\boxed{2}} \times 11^{\boxed{6}} \times 13^{\boxed{10}} \times 17^{\boxed{3}} \times 19^{\boxed{7}}$$

$$\text{“gödel”} = 2^{1+7} \times 3^{1+15} \times 5^{1+4} \times 7^{1+5} \times 11^{1+12} = 139,869,560,310,664,817,087,943,919,200,000$$

- ❖ 若果真如RAM模型所假设的**字长无限**，则只需**一个**寄存器即可...



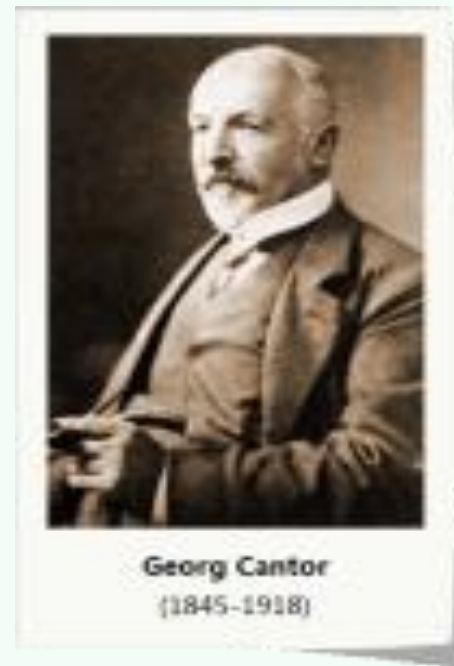
凡物皆数: Cantor Numbering

$$\text{cantor}_2(i, j) = [(i + j)^2 + 3 \cdot i + j] / 2$$

$$\text{cantor}_2(2, 3) = [(2 + 3)^2 + 3 \cdot 2 + 3] / 2 = 17$$

$$\text{cantor}_2(3, 2) = [(3 + 2)^2 + 3 \cdot 3 + 2] / 2 = 18$$

0	1	3	6	10	15
2	4	7	11	16	
5	8	12	17		
9	13	18			
14	19				
20					



$$\text{cantor}_{n+1}(a_1, \dots; a_{n-1}, a_n, a_{n+1}) =$$

$$\text{cantor}_n(a_1, \dots; a_{n-1}, \text{cantor}_2(a_n, a_{n+1}))$$

凡物皆数: Cantor Numbering

❖ 长度**有限**的字符串, 都可视作 $d=1+|\Sigma|$ 进制的自然数

"decade" = $453145_{(10)}$

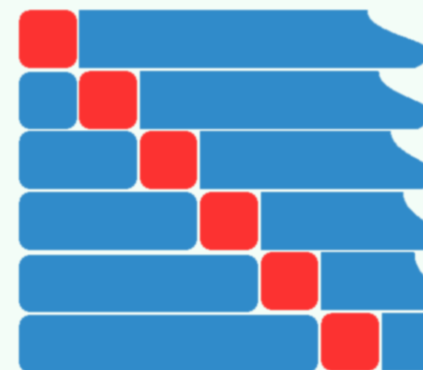
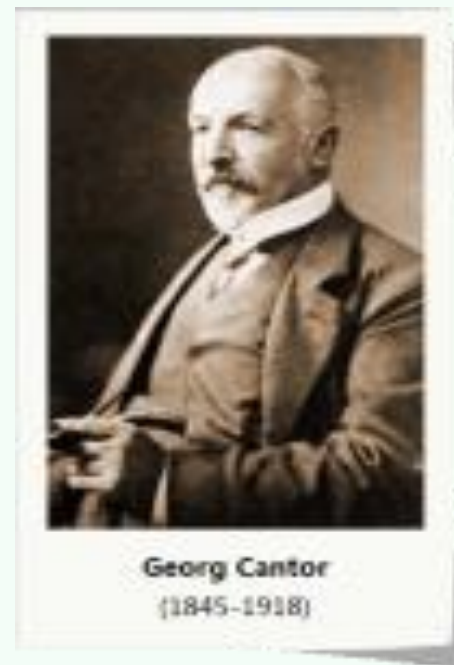
// $d = 1 + ('i' - 'a') = 10$

❖ 长度**无限**的字符串, 都可视作 $[0,1)$ 内的 d 进制小数

"bgahbhahbhdei..." = $0.2718281828459\dots$

"fade" = $0.614\underline{5}$ = $0.614\underline{4999\dots}$ = "fadiiii..."

❖ Cantor's Diagonal: $\aleph_0 < \aleph_1$



串亦为数

❖ 十进制串，可直接视作自然数 //指纹 (fingerprint) , 等效于多项式法

$P = "82818"$

$T = "271\boxed{82818}284590452353602874713527"$

❖ 一般地，随意对字符编号 $\{ 0, 1, 2, \dots, d - 1 \}$ //设 $d = |\Sigma|$

于是，每个字符串都对应于一个 d 进制自然数 //尽管不是单射

$"CAT" = 2 \ 0 \ 19_{(26)} = 1371_{(10)} // \Sigma = \{ A, B, C, \dots, Z \}$

$"ABBA" = 0 \ 1 \ 1 \ 0_{(26)} = 702_{(10)}$

❖ P 在 T 中出现 仅当 T 中某一子串与 P 相等 //为什么不是 “当” ?

❖ 这，不已经就是一个算法了吗？！ //具体如何实现？

❖ 问题似乎解决得很顺利，果真如此简单吗？ //复杂度？

串

Karp-Rabin算法：散列

13-F2

邓俊辉

deng@tsinghua.edu.cn

我有些明白了：如果把要指明的恒星与周围恒星的相对位置信息发送出去，接收者把它与星图进行对照，就确定了这颗恒星的位置

数位溢出

❖ 如果 $|\Sigma|$ 很大，模式串 P 较长，其对应的指纹将**很长**

比如，若将 P 视作 $|P|$ 位的 $|\Sigma|$ 进制**自然数**，并将其作为指纹...

❖ 仍以ASCII字符集为例 $// |\Sigma| = 128 = 2^7$

只要 $|P| > 9$ ，则指纹的长度将至少是： $7 \times 10 = 70$ bits

❖ 然而，目前的字长一般也不过64位 $//$ 存储不便

❖ 而更重要地，指纹的计算与比对，将不能在 $\mathcal{O}(1)$ 时间内完成 $//$ RAM!?

准确地说，需要 $\Omega(|P|/64) = \Omega(m)$ 时间；总体需要 $\mathcal{O}(n \cdot m)$ 时间 $//$ 与蛮力算法相当

❖ 有何高招？

散列压缩：通过**散列**，将指纹压缩至存储器支持的范围

❖ 经过一次比对，指纹**不同**的对齐位置，自然可予**排除**

❖ 比如，采用模余函数： $\text{hash}(\text{key}) = \text{key} \% 97$

P = 8 2 8 1 2 //hash(82818) = 77

T = 2 7 1 8 2 8 1 8 2 8 4 5 9 0 4 5 2 3 5 3 6

2 7 1 8 2 //22

7 1 8 2 8 //48

1 8 2 8 1 //45

8 2 8 1 8 //77

❖ 然而，指纹**相同**，原串却**未必**匹配...

散列冲突：hash()相等时，还须进一步地严格比对

❖ 为此仍需 $\Omega(m)$ 时间，而且

既然是散列压缩，指纹冲突就在所难免

P = 1 8 2 8 4 //hash(18284) = 48

T = 2 7 1 8 2 8 1 8 2 8 4 5 9 0 4 5 2 3 5 3 6

2 7 1 8 2 //22

7 1 8 2 8 //48

...

1 8 2 8 4 //48

❖ 好在，只要散列函数选取得当，便可将冲突的概率控制在可以接受的范围

快速指纹计算

❖ 每次计算`hash()`，均需 $\Omega(m)$ 时间

有可能加速吗？

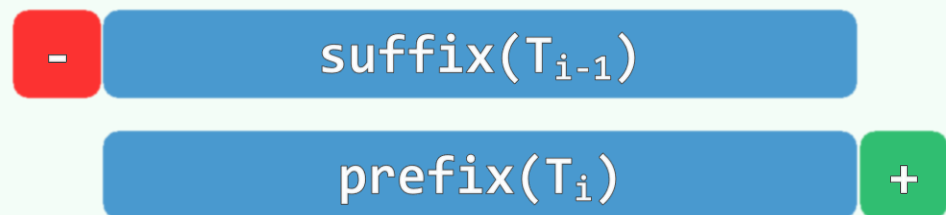
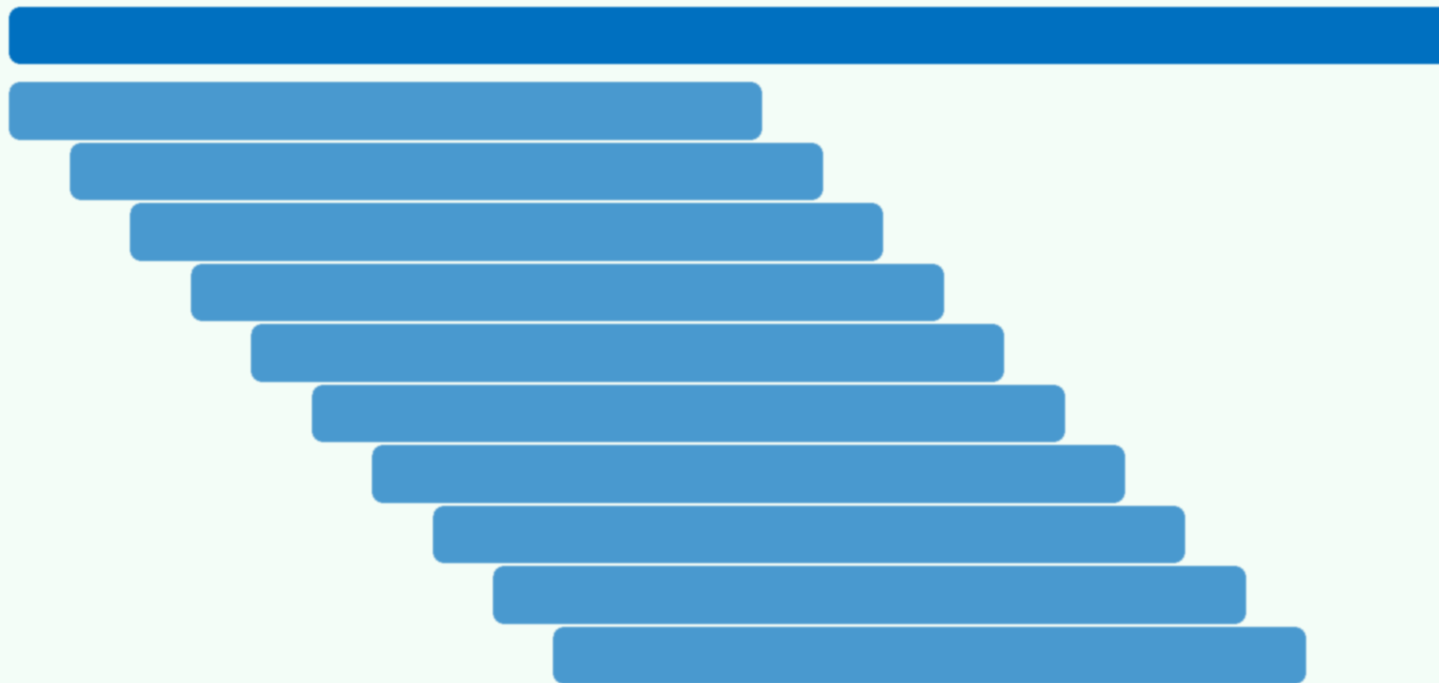
❖ 回忆一下，进制转换算法...

❖ 观察

- 相邻的散列之间，存在极强的相关性
- 相邻的指纹之间，也有极强的相关性

❖ 利用上述性质，即可在 $\mathcal{O}(1)$ 时间内

由上一指纹得到下一指纹...



串

Karp-Rabin算法：位宽

13-F3

God kisses the finite in his love and man the infinite.

当一个人反复思考的时候，就必定会出现悖论，然而不管你们会怎么说，
我宁愿做一个持有悖论的人，也不愿做心存偏见的人

邓俊辉

deng@tsinghua.edu.cn

$$\text{power}_a(n) = a^n$$

```
uint64_t power(uint64_t a, int n) {  
    uint64_t pow = 1;  
    while ( 0 < n ) {  
        pow *= a;  
        n--;  
    }  
    return pow;  
} //蛮力版
```

❖ $T(n) = 1 + 2n = \mathcal{O}(n)$ <-- 线性? 伪线性!

❖ 所谓输入**规模**

准确地应定义为

用以**描述**输入所需的**空间规模**

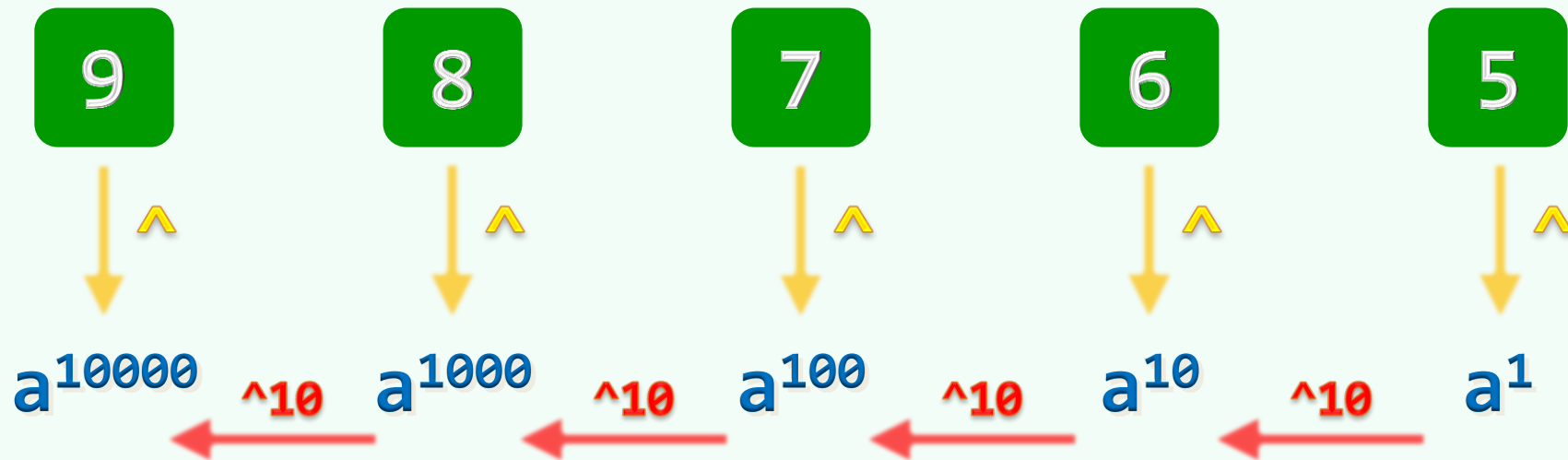
❖ 对于此类数值计算，即是

- n 的**位宽**|bit width
- n 的**二进制位数**，或打印的**宽度**

$$r = \lceil \log_2(n + 1) \rceil = \mathcal{O}(\log n)$$

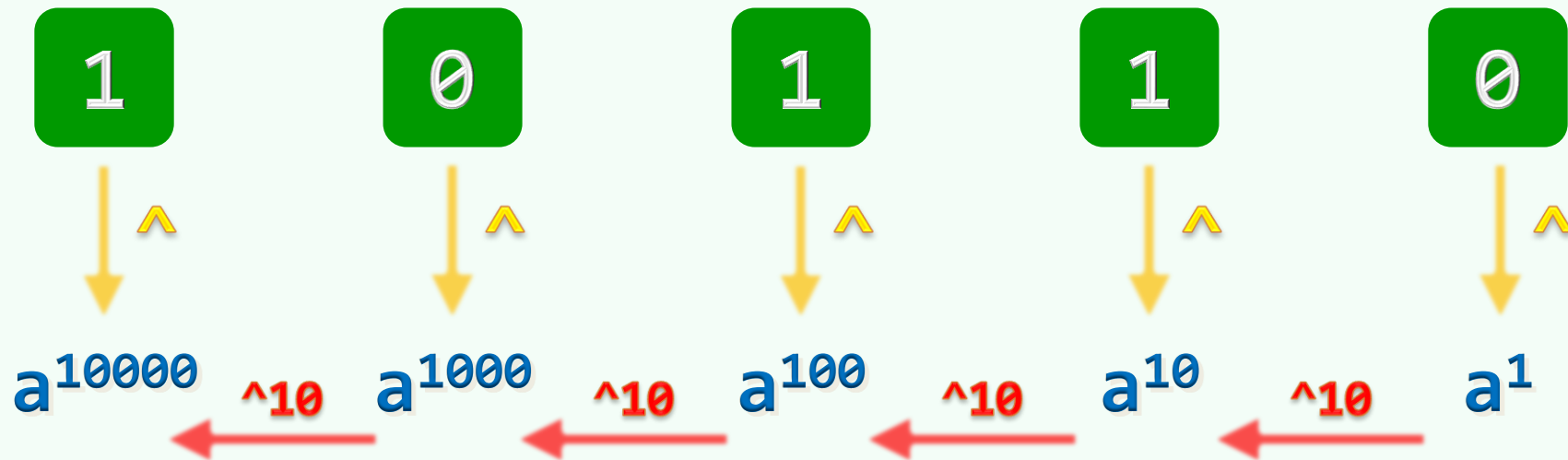
$$T(r) = \mathcal{O}(2^r) \text{ // 指数复杂度!}$$

$$a^{9 \cdot 10^4} + 8 \cdot 10^3 + 7 \cdot 10^2 + 6 \cdot 10^1 + 5 \cdot 10^0$$



$$\left(a^{10^4}\right)^9 \cdot \left(a^{10^3}\right)^8 \cdot \left(a^{10^2}\right)^7 \cdot \left(a^{10^1}\right)^6 \cdot \left(a^{10^0}\right)^5$$

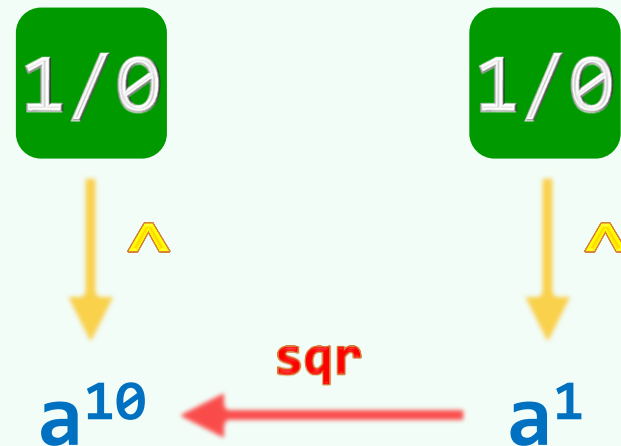
$$a^{1 \cdot 2^4} + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$



$$\left(a^{2^4}\right)^1 \cdot \left(a^{2^3}\right)^0 \cdot \left(a^{2^2}\right)^1 \cdot \left(a^{2^1}\right)^1 \cdot \left(a^{2^0}\right)^0$$

从 $O(n)$ 到 $O(r = \log n)$

```
uint64_t power(uint64_t a, int n) {  
    uint64_t pow = 1, p = a;  
    while (0 < n) { //O(logn)  
        if (n & 1) //O(1)  
            pow *= p; //O(1)  
        n >>= 1; //O(1)  
        p *= p; //O(1)  
    }  
    return pow; //O(1)  
} //高效版
```



❖ 输入规模 $= r = \lceil \log_2 (n + 1) \rceil$

❖ 运行时间 $= T(r) = 1 + 1 + 4r + 1 = O(r)$

❖ 如此, “实现”了从指数到线性的改进

悖论?

❖ 根据以上算法, “可以”

在 $\mathcal{O}(\log n)$ 时间内计算出 a^n

❖ 然而, a^n 的二进制展开宽度为 $\Theta(n)$

即便是直接打印, 也需要 $\Omega(n)$ 时间

...哪里...错了?

❖ 类似的悖论, 对 $\text{fib}(n)$ 也存在...

仿照以上`power()`算法, 也 “可以”

在 $\mathcal{O}(\log n)$ 时间内计算出 $\text{fib}(n)$

❖ 令: $\mathcal{A} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} \text{fib}(0) & \text{fib}(1) \\ \text{fib}(1) & \text{fib}(2) \end{bmatrix}$

则: $\mathcal{A}^n = \begin{bmatrix} \text{fib}(n-1) & \text{fib}(n) \\ \text{fib}(n) & \text{fib}(n+1) \end{bmatrix}$

❖ RAM模型: 两种代价准则

- 常数 | uniform cost criterion
- 对数 | logarithmic cost criterion

13-G

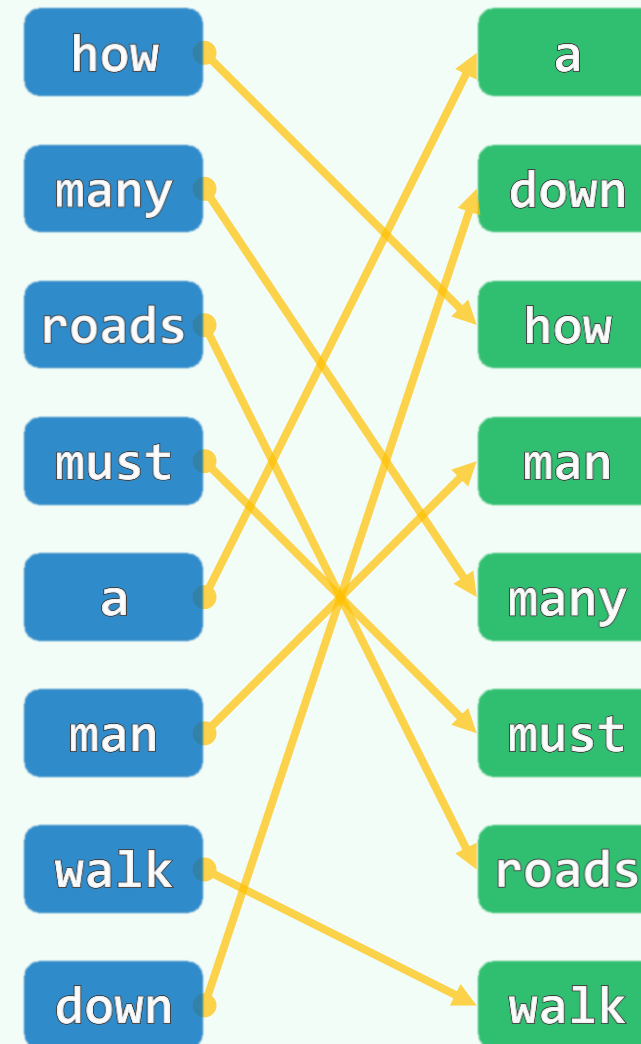
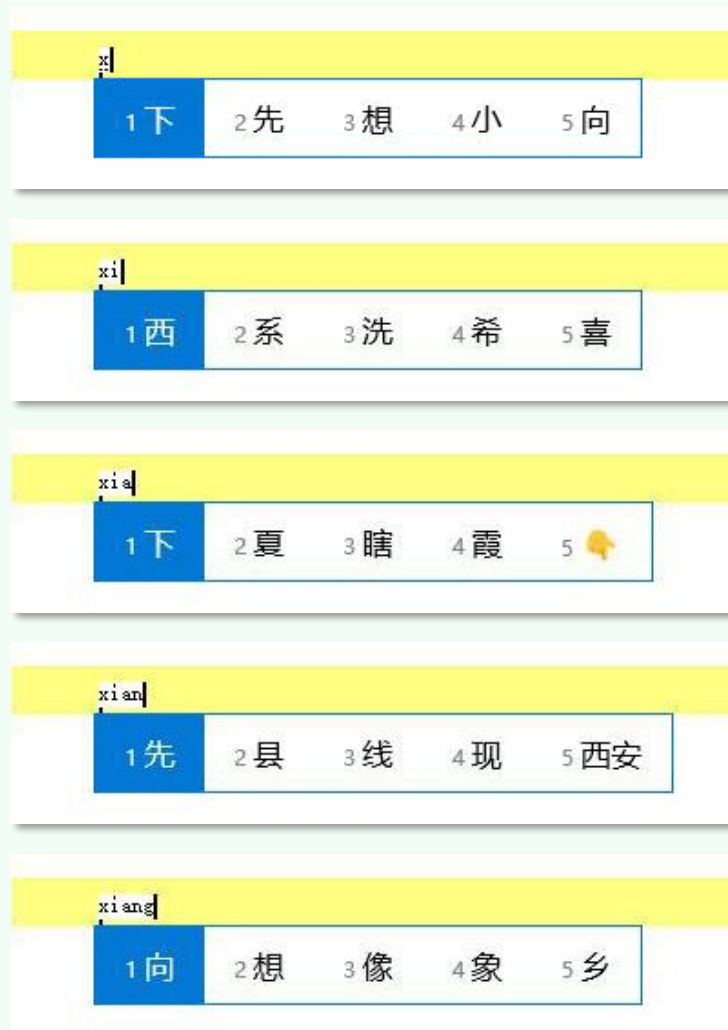
串
键树

兩個吃罷飯，又走了四五十里，卻來到一市鎮上，地名喚做瑞龍鎮，卻是個三岔路口。宋江借問那裏人道：“小人們欲投二龍山、清風鎮上，不知從那條路去？”

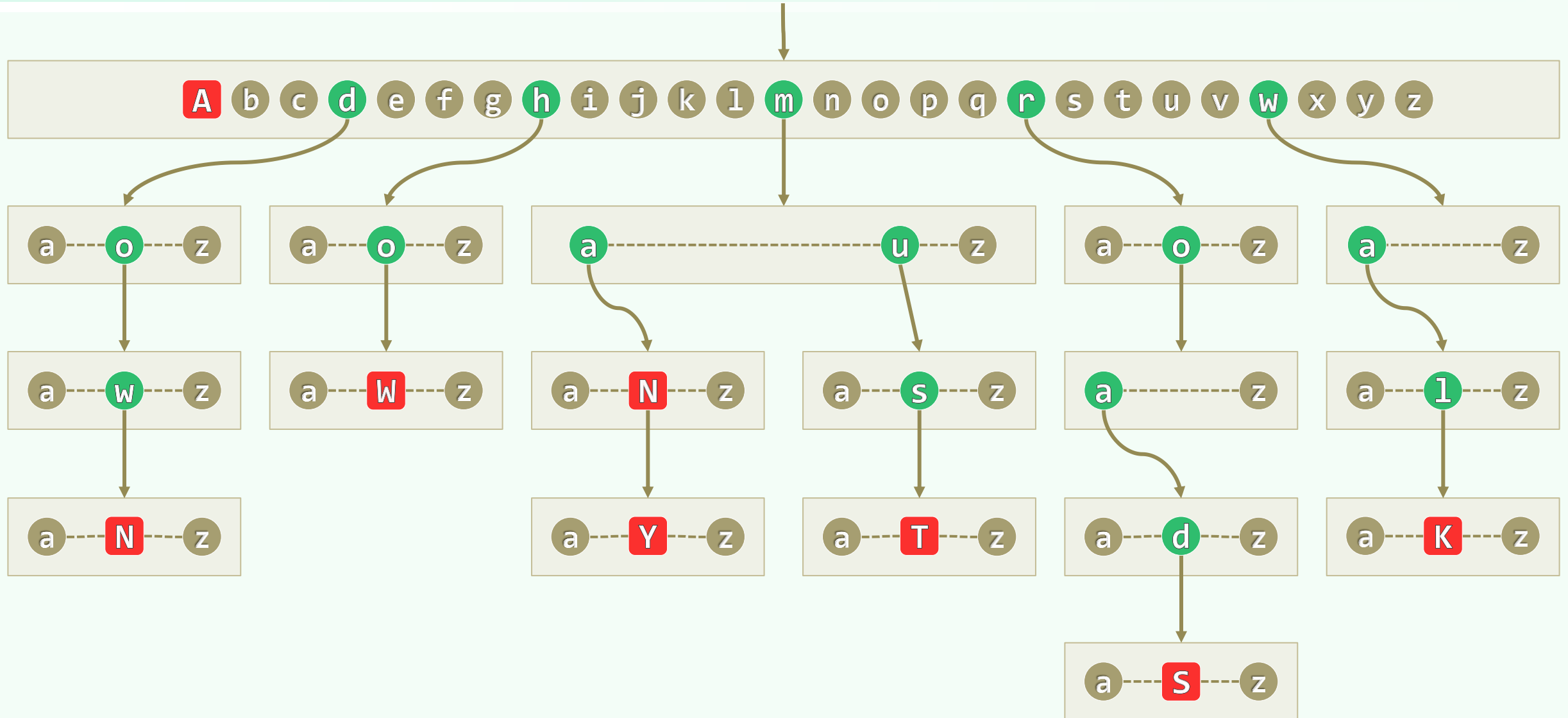
邓俊辉

deng@tsinghua.edu.cn

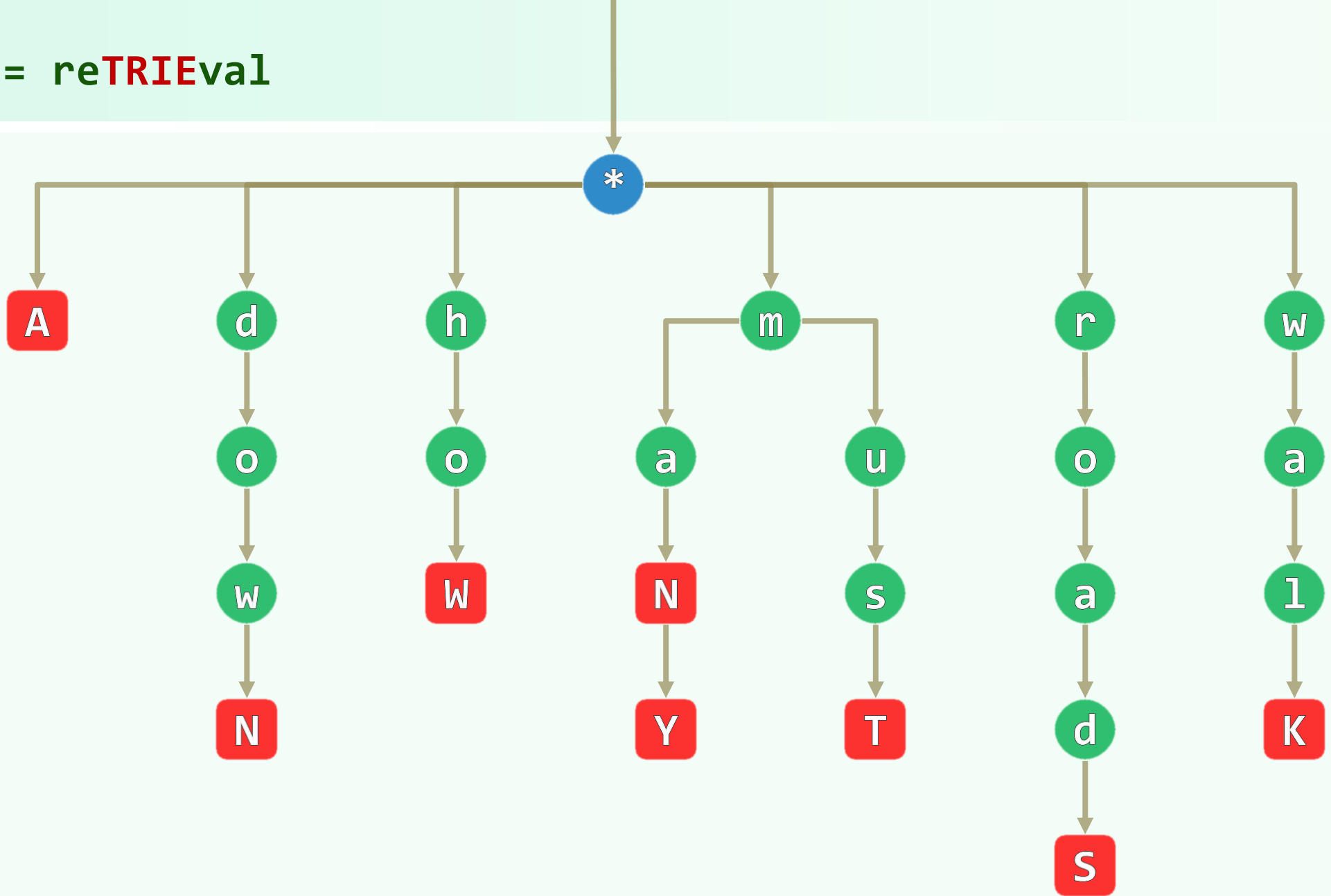
Trie = Digital Tree = Radix Tree = Prefix Tree



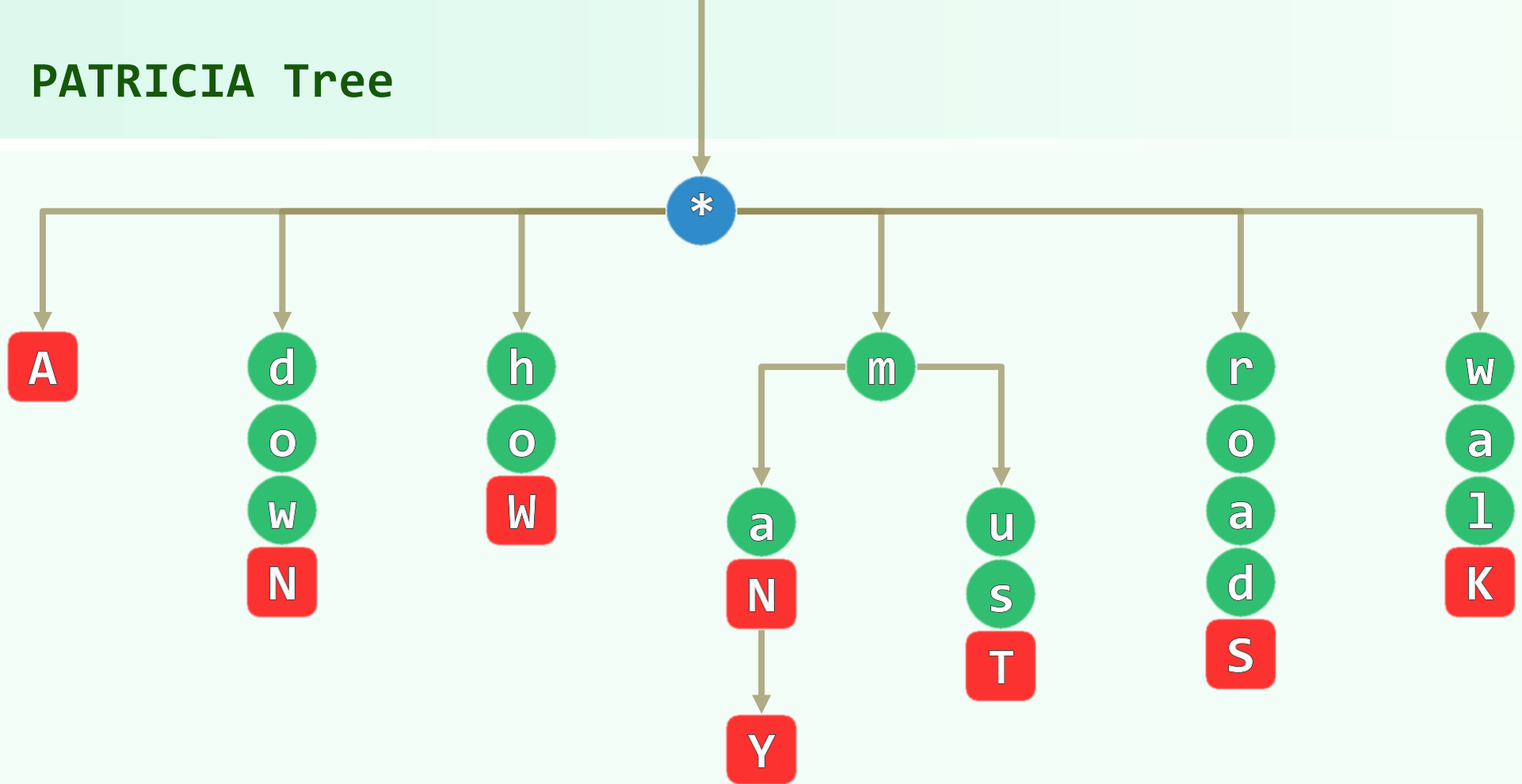
Trie: how many roads must a man walk down



Trie = reTRIEval



Trie: PATRICIA Tree



Trie: Ternary Trie

